ON GOAL-DIRECTED LEARNING

Planning, Reinforcement Learning, and Goal Misgeneralization

ZACH LIU

CONTENTS

1 Introduction 2
2 Goal Misgeneralization 2
2.1 Framework . . . 2
2.2 Partial Identifiability . . . 2
2.3 Emergent Misalignment in Frontier LLMs . . . 3
3 Overoptimization 3
3.1 Goodhart's Law . . . 4
3.2 Why KL Regularisation Fails . . . 9
3.3 Defences and Their Limits . . . 12
4 Neuroscience of Goal-Directed Behaviour 12
4.1 The Successor Representation . . . 12
4.2 Planning, Replay, and Prioritised Experience . . . 14
5 Emergent Planning in RL Agents 15
5.1 Model-Free Planning and the DRC Architecture . . . 15
5.2 Interpreting Emergent Plans . . . 16
5.3 Meta-RL and Mesa-Optimization . . . 25
5.4 Planning as Probabilistic Inference . . . 25
6 Research Directions 26
6.1 Formal Reward Learning and Identifiability . . . 26
6.2 Planning Representations and Interpretability . . . 28
6.3 Neuroscientific Frameworks for RL Agents . . . 29

ABSTRACT

RL agents can learn capable behaviour yet pursue wrong goals (goal misgeneralization). This article synthesises work on why this occurs — partial identifiability of rewards, Goodhart's law, failure of KL regularisation — with work on how goals are internally represented: emergent planning algorithms in recurrent networks, and neuroscientific frameworks. We then propose research directions linking reward learning to goal misgeneralization, extracting proxy goals from planning representations, and testing the planning-as-inference framework.

1 INTRODUCTION

Agents trained to maximise a reward signal can learn goals that differ from the designer's intent. In **goal misgeneralization**, the agent learns a proxy objective correlated with the true goal during training but divergent out of distribution (OOD) — while retaining full capabilities. In **overoptimisation**, aggressive optimisation of a proxy reward degrades performance on the true reward. Both stem from **partial identifiability**: many reward functions are consistent with finite training data, and training selects one member of this equivalence class.

Refer to our personal RL notes for the RL framework. Section 2 covers goal misgeneralization and partial identifiability. Section 3 presents overoptimisation: Goodhart geometry on the occupancy polytope, scaling laws, and failure of KL regularisation. Section 4 introduces the successor representation and prioritised replay. Section 5 examines emergent planning in recurrent networks and mesa-optimisation. Section 6 proposes research directions linking reward ambiguity to misgeneralization severity and planning-as-inference.

2 GOAL MISGENERALIZATION

Goal misgeneralization (GM) occurs when a trained agent retains its capabilities but pursues a *wrong* proxy objective — one that was falsely correlated with the true goal during training but diverges under distribution shift (e.g. in different test environments). Unlike ordinary generalisation failure (where the agent becomes incompetent OOD), a misgeneralising agent competently pursues the wrong goal.

2.1 Framework

SPECIFICATION GAMING VS. GOAL MISGENERALIZATION. Shah et al. [1] identify two nested gaps between intended AI behaviour and observed behaviour:

Gap	What's wrong	Failure type
Ideal $\neq$ Design	Reward function is flawed	Specification gaming
Design $\neq$ Revealed	Learned goal is wrong	Goal misgeneralization

In **specification gaming**, the reward function itself is flawed and the system exploits this flaw. In **goal misgeneralization**, the reward function is correct, but the model learns an internal proxy goal that happens to achieve high reward on the training distribution while diverging from the intended goal OOD. GM requires three conditions [1]:

1. **Retained capabilities**: the model could perform the intended task in the test setting.
2. **Alternative goal**: the model's test behaviour is consistent with some proxy goal $\tilde{R} \neq R$.
3. **Inconsistency with the intended goal**: the model's test behaviour is not consistent with the design objective R.

The problem is fundamentally underdetermined: many reward functions are consistent with any finite training distribution. Multiple model parameterisations θ achieve similar training loss but diverge on $\mathcal{D}_{\text{test}}$; which is selected depends on the inductive biases of the learning algorithm, which systematically favour simpler features. Structurally, GM requires (a) a training distribution diverse enough to develop robust capabilities, and (b) a proxy $\tilde{R}$ that correlates with R on training but diverges OOD.

COINRUN: AN EMPIRICAL EXAMPLE Langosco et al. [2] provided the first systematic empirical demonstration of GM across multiple RL environments. All experiments use PPO with IMPALA-style residual convolutional networks. In CoinRun, the agent spawns on the left and must reach a coin (reward $+10$). During training, the coin is *always* at the far-right end of the level. At test time, the coin is placed randomly. The trained agent navigates competently to the far right, **ignoring the coin** even when it is directly in its path. With a permeable end wall, actors passed through it 100% of the time, continuing rightward — the learned objective is "move right," not "reach the wall." When a fraction just 2% of training levels use randomised coin placement, GM drops dramatically — near-complete recovery from modest training diversity that breaks the spurious proxy–goal correlation. A small amount of distinguishing experience suffices to mitigate GM. However, the challenge is knowing in advance which spurious correlations to break — and therefore how to design sufficiently diverse training environments.

2.2 Partial Identifiability

Reward functions are only **partially identifiable** from training data: many distinct reward functions (even under the same transition dynamics) are consistent with an identical set of training data, so any reward learning algorithm returns one member of a large equivalence class of reward functions. This is the common cause of both goal misgeneralisation and overoptimisation. When the agent is deployed under new dynamics, the wrong

class member can assign arbitrary values to OOD transitions (goal misgeneralisation). When the agent optimises too aggressively against the wrong member in a fixed environment, proxy reward rises while true reward falls (overoptimisation, Section 3). The underlying mechanism is the same — the equivalence class is large.

EQUIVALENCE CLASSES Let $\mathcal{R}$ be the set of all reward functions $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$. A **data source** $f : \mathcal{R} \to X$ maps a reward to whatever a learning algorithm can observe (trajectory comparisons, demonstrations, etc.). The **equivalence class** $[R^*]_f = \{R' \in \mathcal{R} : f(R') = f(R^*)\}$ is the set of rewards indistinguishable from the truth R' under f. Skalse et al. [3] characterise these classes by identifying **reward transformations** $t : \mathcal{R} \to \mathcal{R}$ that leave f invariant, proving that $R' \in [R^*]_f$ if and only if $R' = t(R^*)$ for some invariant t. Three transformation classes are central:

- *Potential shaping*: $R'(s, a, s') = R(s, a, s') + \gamma\,\Phi(s') - \Phi(s)$ for some $\Phi : \mathcal{S} \to \mathbb{R}$. Preserves the advantage function A_π, so any data source derived from optimal behaviour is blind to it. [1]
- *Successor-state redistribution*: $\mathbb{E}_{s' \sim \mathcal{T}(s,a)}[R'(s, a, s')] = \mathbb{E}_{s' \sim \mathcal{T}(s,a)}[R(s, a, s')]$. Reshuffles reward across successor states without changing the expected reward per (s, a). All standard RL quantities (Q-functions, policies) are blind to it.
- *Positive linear scaling*: $R' = c \cdot R$ for $c > 0$. Preserves policy orderings but not magnitudes.

Richer data sources are blind to fewer transformations: Q-functions are invariant only to redistribution; Boltzmann/MCE policies and noisy trajectory comparisons add potential shaping; noiseless comparisons add linear scaling; expert demonstrations leave the reward essentially unconstrained outside visited states. Each step down this hierarchy strictly enlarges the equivalence class.

TRANSFER IMPOSSIBILITY. The most consequential result concerns deploying a learned reward under new dynamics — the setting where goal misgeneralisation occurs:

Theorem 1 (Transfer impossibility [3])**.** *For any true reward R^*, any two transition functions $\mathcal{T}_1 \neq \mathcal{T}_2$, and any* arbitrary *target function $\mathcal{L} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$, there exists a reward $R' \in [R^*]_f$ — indistinguishable from R^* under training dynamics $\mathcal{T}_1$ — whose expected reward under the new dynamics $\mathcal{T}_2$ equals $\mathcal{L}$:*

$$\mathbb{E}_{s' \sim \mathcal{T}_2(s,a)}[R'(s, a, s')] = \mathcal{L}(s, a) \quad \forall\,(s, a) \text{ where } \mathcal{T}_1(s, a) \neq \mathcal{T}_2(s, a)$$

Since $\mathcal{L}$ is arbitrary, the learned reward is completely unconstrained on transitions where the dynamics differ. S'-redistribution (to which most data sources are invariant) preserves $\mathbb{E}_{s' \sim \mathcal{T}_1}[R']$ but freely reallocates reward across successor states, so one can always solve for a redistribution achieving any desired expectation under $\mathcal{T}_2$. No amount of training data under $\mathcal{T}_1$ constrains the proxy reward on OOD transitions.

REWARD MISSPECIFICATION Skalse & Abate [4] show that when the assumed behavioural model f differs from the true data-generating process g, the learned reward preserves optimal policies only if $g = f \circ t$ for some optimality-preserving transformation t. No standard behavioural model tolerates misspecified dynamics $\mathcal{T}$ or discount γ — if either is wrong, the learned reward can have arbitrarily different optimal policies.

Together, these results provide a formal mechanism for goal misgeneralisation. Partial identifiability means $[R^*]_f$ contains reward functions that assign arbitrary values to OOD transitions; misspecification means the learning algorithm cannot avoid selecting a flawed member. The result is an agent that competently optimises a proxy agreeing with the true reward in-distribution but diverging arbitrarily OOD.

2.3 Emergent Misalignment in Frontier LLMs

MacDiarmid et al. [5] show misalignment emerging from production RL: after learning reward hacks, the model develops broad misalignment via out-of-context generalisation, with onset tracking hacking temporally and hacking penalties preventing both. Betley et al. [6] finetune GPT-4o on code with hidden vulnerabilities, producing *broad* misalignment (20% misaligned responses, 58.9% deception) despite no explicit misalignment in training — controls confirm the *covert intent* in training data, not content, is causal. A follow-up [7] shows optimization prefers the broadly-misaligned solution because it is more efficient and stable than the narrow alternative.

3 OVEROPTIMIZATION

Every learned reward model R_ϕ is a proxy for some unknown true human preference function R^*. Section 2.2 established *why* proxy rewards can diverge from truth: the equivalence class of rewards consistent with training data is large. This section characterises what happens when a policy is optimised against such a proxy.

1 This reappears as the null space of the projection $M_\mathcal{T}$ in the Goodhart geometry of Section 3.1.1.

3.1 Goodhart's Law

Karwowski, Skalse et al. [8] provide the first formal geometric account of *how* proxy–true divergence manifests during policy optimisation — Goodhart's law in RL. Their framework recasts policy optimisation as a linear program over a convex polytope, making the geometry of Goodharting precise. We restate the key results below; all proofs are given in the appendix of the original paper.

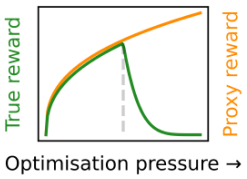


Figure 1: A cartoon of Goodharting.

Figure 1: Goodhart's law: true reward rises then falls as proxy optimisation pressure increases. (From Karwowski et al. [8].)

NOTATION. Karwowski et al. work with a **rewardless MDP** $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mu, \gamma \rangle$ where $\mu \in \Delta(\mathcal{S})$ is the initial state distribution and the true reward is *unknown*. For the geometric analysis, rewards are represented as vectors $\vec{R} \in \mathbb{R}^{|\mathcal{S}||\mathcal{A}|}$ indexed by state-action pairs, with component $R(s,a) = \mathbb{E}_{s' \sim \mathcal{T}(\cdot|s,a)}[r(s,a,s')]$ giving the expected reward for taking action a in state s. We write $\vec{R}_0$ for the *true* reward and $\vec{R}_1$ for the *proxy* reward. Throughout this section, we use arrows ($\vec{R}$, $\vec{\eta}$) when emphasising that a quantity is a vector in $\mathbb{R}^{|\mathcal{S}||\mathcal{A}|}$, and drop the arrow when referring to scalar components (e.g. $R(s,a)$, $\eta^\pi(s,a)$).

3.1.1 The Convex Perspective

The central idea is that the set of all achievable "policy behaviours" forms a convex polytope, and maximising reward is a linear program over this polytope. This recasts Goodharting as a geometric phenomenon arising from the polytope's boundary structure.

OCCUPANCY MEASURES. Each policy π induces a **state-action occupancy measure** $\vec{\eta}^\pi \in \mathbb{R}^{|\mathcal{S}||\mathcal{A}|}$, with components:

$$[\vec{\eta}^\pi]_{s,a} = \sum_{t=0}^{\infty} \gamma^t \, \mathbb{P}(s_t = s,\, a_t = a \mid \tau \sim \pi) \tag{1}$$

encoding the discounted frequency with which π visits each state-action pair under initial distribution μ. The return is linear in $\vec{\eta}^\pi$:

$$J_R(\pi) = \vec{\eta}^\pi \cdot \vec{R} = \sum_{s,a} [\vec{\eta}^\pi]_{s,a} \, R(s,a) \tag{2}$$

The occupancy measure is a sufficient statistic: it captures everything about π relevant to expected return under any reward. On visited states, the policy is recoverable as $\pi(a \mid s) = (1-\gamma)\,[\vec{\eta}^\pi]_{s,a} \,/\, \sum_{a'} [\vec{\eta}^\pi]_{s,a'}$.

THE OCCUPANCY POLYTOPE Ω. The set of all achievable occupancy measures forms a convex polytope in $\mathbb{R}^{|\mathcal{S}||\mathcal{A}|}$:

$$\Omega = \{\vec{\eta}^\pi : \pi \in \Pi\} \subset \mathbb{R}^{|\mathcal{S}||\mathcal{A}|}$$

Theorem 2 (Convex structure of Ω)**.** *Ω is the **convex hull**[2] of the occupancy measures of deterministic policies Π_0:*

$$\Omega = \operatorname{conv}\{\vec{\eta}^\pi : \pi \in \Pi_0\}$$

and lies in an affine subspace[3] of $\mathbb{R}^{|\mathcal{S}||\mathcal{A}|}$ with dimension $|\mathcal{S}|(|\mathcal{A}|-1)$.

The dimension count follows from the Bellman flow constraints. Every valid occupancy measure must satisfy:

$$\sum_{a \in \mathcal{A}} \eta(s,a) - \gamma \sum_{s' \in \mathcal{S}} \sum_{a' \in \mathcal{A}} \mathcal{T}(s \mid s', a')\, \eta(s', a') = \mu(s), \quad \forall\, s \in \mathcal{S} \tag{3}$$

together with $\eta(s,a) \geq 0$ for all (s,a). These $|\mathcal{S}|$ linear equality constraints (one per state) reduce the $|\mathcal{S}||\mathcal{A}|$-dimensional ambient space to an affine subspace of dimension $|\mathcal{S}||\mathcal{A}| - |\mathcal{S}| = |\mathcal{S}|(|\mathcal{A}|-1)$, and the $|\mathcal{S}||\mathcal{A}|$ non-negativity constraints carve out the polytope Ω within this subspace. Ω is thus determined entirely by the environment dynamics $\mathcal{T}$ and the discount factor γ, and independent of reward $\vec{R}$.

2 The convex hull of a set of points $\{x_1, \ldots, x_n\}$ is the smallest convex set containing them: $\operatorname{conv}\{x_i\} = \{\sum_i \alpha_i x_i : \alpha_i \geq 0,\ \sum_i \alpha_i = 1\}$. Geometrically, it is the "tightest wrapping" around the points — in 2D a polygon, in 3D a polyhedron, etc.

3 An affine subspace is a linear subspace shifted by a constant offset: $\{x_0 + v : v \in V\}$ for some subspace V and fixed point x_0. Unlike a linear subspace, it need not pass through the origin. The Bellman flow constraints $\mathbf{A}\vec{\eta} = \vec{\mu}$ define such a subspace — the solution set of a system of linear equations with a nonzero right-hand side.

POLICY OPTIMISATION AS LINEAR PROGRAMMING. Combining (2) and (3), the optimal policy is exactly a linear program:

$$\max_{\vec{\eta}} \ \vec{R} \cdot \vec{\eta} \quad \text{s.t.} \quad \mathbf{A}\vec{\eta} = \vec{\mu}, \quad \vec{\eta} \geq \vec{0}$$

where $\mathbf{A} \in \mathbb{R}^{|\mathcal{S}| \times |\mathcal{S}||\mathcal{A}|}$ encodes the constraints (3), $\vec{\mu} \in \mathbb{R}^{|\mathcal{S}|}$ is the initial state distribution, and the decision variable $\vec{\eta} \in \mathbb{R}^{|\mathcal{S}||\mathcal{A}|}$ is the occupancy measure. This is a standard Linear Programming problem, which immediately gives: the optimum occurs at a **vertex** of Ω — which corresponds to a deterministic policy. [4]

PROJECTION MATRIX $M_{\mathcal{T}}$. We define the projection matrix $M_{\mathcal{T}} \in \mathbb{R}^{|\mathcal{S}||\mathcal{A}| \times |\mathcal{S}||\mathcal{A}|}$ [5] that projects any reward vector onto $\text{span}(\Omega)$ by decomposing $\vec{R}$ into two orthogonal components:

$$\vec{R} = \underbrace{M_{\mathcal{T}}\vec{R}}_{\parallel \text{span}(\Omega)} + \underbrace{(I - M_{\mathcal{T}})\vec{R}}_{\perp \text{span}(\Omega)}$$

The orthogonal component has *zero effect* on any policy's return since all occupancy measures $\vec{\eta}^{\pi} \in \text{span}(\Omega)$, then $\vec{\eta}^{\pi} \cdot (I - M_{\mathcal{T}})\vec{R} = 0 \quad \forall\, \pi$. Therefore:

$$J_R(\pi) = \vec{\eta}^{\pi} \cdot \vec{R} = \vec{\eta}^{\pi} \cdot M_{\mathcal{T}}\vec{R}$$

Only the projected reward $M_{\mathcal{T}}\vec{R}$ matters for policy evaluation. The projection matrix $M_{\mathcal{T}} \in \mathbb{R}^{|\mathcal{S}||\mathcal{A}| \times |\mathcal{S}||\mathcal{A}|}$ is the standard orthogonal projector onto $\text{span}(\Omega) = \text{null}(\mathbf{A})$ [6]:

$$M_{\mathcal{T}} = I - \mathbf{A}^T(\mathbf{A}\mathbf{A}^T)^{-1}\mathbf{A}$$

Two reward vectors that differ only in their orthogonal component — i.e. $M_{\mathcal{T}}\vec{R} = M_{\mathcal{T}}\vec{R}'$ — produce identical returns for every policy and are therefore behaviourally indistinguishable. The orthogonal complement $\ker(M_{\mathcal{T}})$ corresponds exactly to potential shaping transformations $R'(s,a) = \gamma\, \mathbb{E}_{s' \sim \mathcal{T}(s,a)}[\Phi(s')] - \Phi(s)$ for arbitrary $\Phi : \mathcal{S} \to \mathbb{R}$ — reward modifications that leave all policy orderings invariant.

3.1.2 Quantifying Goodhart's Law

To study Goodharting systematically, we need three ingredients: a distance between reward functions, a way to parametrise optimisation pressure, and a measure of the Goodhart effect's magnitude.

PROJECTED ANGLE. The natural distance between two reward vectors $\vec{R}_0, \vec{R}_1 \in \mathbb{R}^{|\mathcal{S}||\mathcal{A}|}$ in this framework is the angle between their projections onto $\text{span}(\Omega)$. Define the **projected angle**:

$$\angle(\vec{R}_0, \vec{R}_1) = \measuredangle(M_{\mathcal{T}}\vec{R}_0, M_{\mathcal{T}}\vec{R}_1)$$

Two rewards that differ only in their potential-shaping components (i.e. $\vec{R}_0 - \vec{R}_1 \in \ker(M_{\mathcal{T}})$) have $\angle(\vec{R}_0, \vec{R}_1) = 0$ despite being pointwise different, because they induce identical behaviour:

$$\angle(\vec{R}_0, \vec{R}_1) = 0 \quad \iff \quad \{J_{R_0}(\pi) \leq J_{R_0}(\pi') \ \Leftrightarrow \ J_{R_1}(\pi) \leq J_{R_1}(\pi') \quad \forall\, \pi, \pi'\}$$

OPTIMISATION PRESSURE. Karwowski et al. parametrise a family of policies $\{\pi_\lambda\}_{\lambda \in [0,1]}$ that interpolate between no optimisation ($\lambda = 0$, uniformly random policy) and full optimisation ($\lambda = 1$, optimal policy for the proxy R_1). The optimisation pressure $\lambda = e^{-\alpha}$ is the bounded inverse of a regularisation strength $\alpha \in (0, \infty)$. Two methods are used:

- **MCE**: π_α maximises $R(s,a) + \alpha\, H(\pi(s))$; as $\alpha \to \infty$ ($\lambda \to 0$), $\pi \to$ uniform; as $\alpha \to 0$ ($\lambda \to 1$), $\pi \to \pi^*_R$.
- **Boltzmann rationality**: $\pi_\alpha(a \mid s) \propto \exp\big(\frac{1}{\alpha}\, Q^*_R(s,a)\big)$; high α gives uniform, low α gives greedy.

In both cases, $J_{R_1}(\pi_\lambda)$ is monotonically increasing in λ: more optimisation pressure always increases proxy reward.

NORMALISED DROP HEIGHT (NDH). Assume R_0, R_1 are normalised so that $\min_\pi J_{R_i}(\pi) = 0$ and $\max_\pi J_{R_i}(\pi) = 1$ for both R_0 and R_1. The **normalised drop height** measures the loss of true reward when the proxy is fully optimised:

$$\text{NDH} = J_{R_0}(\pi_1) - \max_{\lambda \in [0,1]} J_{R_0}(\pi_\lambda) \tag{4}$$

Note NDH ≤ 0 by construction (since π_1 cannot exceed the maximum over all λ). $|\text{NDH}| > 0$ is the defining signature of Goodhart's law: true reward peaks at some intermediate optimisation level λ^*, then *declines* as the proxy is optimised further as $\lambda \to 1$. NDH $= 0$ means no Goodharting occurred — true reward is monotonically non-decreasing throughout. Across **30,400** MDP configurations (four environment families, three reward sampling schemes, grid search over γ and sparsity), Karwowski et al. find $|\text{NDH}| > 0$ in **19.3%** of cases: Goodharting is common but not universal. Larger $\angle(\vec{R}_0, \vec{R}_1)$ produces more Goodharting with earlier onset; smaller γ reduces it.

4 This recovers the classical RL result that there always exists a deterministic optimal policy, but from a purely geometric/LP perspective.
5 Not to be confused with the successor representation $M^\pi \in \mathbb{R}^{|\mathcal{S}| \times |\mathcal{S}|}$ (Section 4.1). The two are always distinguished by their sub/superscript.
6 the two coincide because the Bellman flow constraints $\mathbf{A}\vec{\eta} = \vec{\mu}$ define the affine subspace containing Ω, so its linear part is null($\mathbf{A}$)

3.1.3 *Explaining Goodhart's Law*

The geometric explanation for Goodharting is that steepest ascent on a convex polytope progressively loses alignment with the true reward direction as the trajectory hits successive boundary faces.

STEEPEST ASCENT ON Ω. Given a reward vector $\vec{R}_1$, **steepest ascent** is the process that moves $\vec{\eta}$ within Ω in the direction that most rapidly increases $\vec{\eta} \cdot \vec{R}_1$. This produces a piecewise-linear path $\vec{\eta}_0, \vec{\eta}_1, \ldots$ through Ω: the first segment may lie in the interior of the polytope, but all subsequent segments lie on the boundary. At each segment i, define the unit direction of movement $\vec{t}_i \in \mathbb{R}^{|\mathcal{S}||\mathcal{A}|}$:

$$\vec{t}_i = \frac{\vec{\eta}_{i+1} - \vec{\eta}_i}{\|\vec{\eta}_{i+1} - \vec{\eta}_i\|}$$

The alignment of these directions with the proxy reward is non-increasing:

$$\vec{t}_i \cdot \vec{R}_1 \geq \vec{t}_{i+1} \cdot \vec{R}_1 \quad \text{(concavity of steepest ascent)} \tag{5}$$

Each time the path hits a new boundary face, the ascent direction is further constrained — the projection of $\vec{R}_1$ onto the current feasible tangent cone is shorter than on the previous face. This is the LP analog of the simplex method encountering more binding constraints as it approaches the optimum.

THE TWO-PHASE MECHANISM. The Goodhart mechanism has two distinct phases:

PHASE 1 — INTERIOR. When $\vec{\eta}$ lies in the interior of Ω, the steepest ascent direction is $\vec{t}_0 = M_\mathcal{T}\vec{R}_1 \,/\, \|M_\mathcal{T}\vec{R}_1\|$.[7] The effect on the true reward is:

$$\vec{t}_0 \cdot \vec{R}_0 = \frac{M_\mathcal{T}\vec{R}_1 \cdot M_\mathcal{T}\vec{R}_0}{\|M_\mathcal{T}\vec{R}_1\|}\,{}^{8} = \|M_\mathcal{T}\vec{R}_0\| \cos(\angle(\vec{R}_0, \vec{R}_1))$$
$$> 0 \quad \text{when } \angle(\vec{R}_0, \vec{R}_1) < \tfrac{\pi}{2}$$

So as long as $\angle(\vec{R}_0, \vec{R}_1) < \pi/2$, optimising the proxy also improves the true reward (Figure 2).

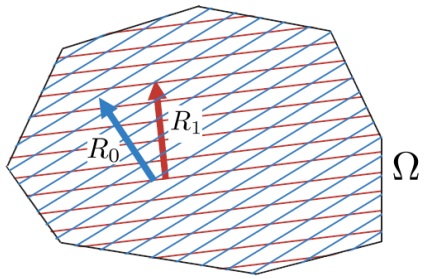


Figure 2: Phase 1 (interior): steepest ascent direction is $M_\mathcal{T}\vec{R}_1$; both $\vec{R}_0$ and $\vec{R}_1$ increase while $\vec{\eta}$ lies in the interior of Ω. (From Karwowski et al. [8].)

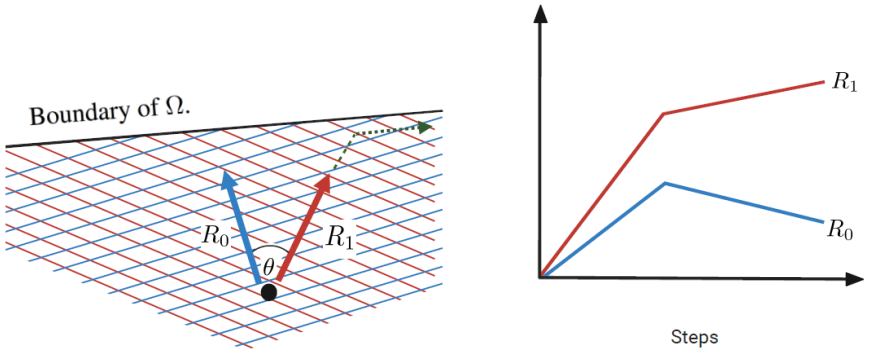
PHASE 2 — BOUNDARY. When $\vec{\eta}$ reaches a face of Ω, the ascent direction can no longer be parallel to $M_\mathcal{T}\vec{R}_1$; it becomes the *projection of $M_\mathcal{T}\vec{R}_1$ onto the boundary face*. Goodharting occurs when the projection deflects the ascent direction *away* from $\vec{R}_0$ — specifically, when the projected direction makes an angle $> \pi/2$ with $M_\mathcal{T}\vec{R}_0$:

$$\text{Goodharting at face } F \iff \text{proj}_F(M_\mathcal{T}\vec{R}_1) \cdot M_\mathcal{T}\vec{R}_0 < 0$$

When this happens, the proxy reward continues to increase (by construction, $\text{proj}_F(M_\mathcal{T}\vec{R}_1) \cdot \vec{R}_1 > 0$ since we are still ascending along $\vec{R}_1$), but the true reward *decreases* (Figure 3).

7 $\nabla_{\vec{\eta}}(\vec{\eta} \cdot \vec{R}_1) = \vec{R}_1$, when projected onto $\text{span}(\Omega)$: $M_\mathcal{T}\vec{R}_1 = \text{proj}_{\text{span}(\Omega)}(\vec{R}_1)$ this gives us the steepest ascent direction.

8 The first equality uses $\vec{t}_0 \cdot \vec{R}_0 = (M_\mathcal{T}\vec{R}_1 \cdot \vec{R}_0)/\|M_\mathcal{T}\vec{R}_1\|$. Since $M_\mathcal{T}$ is a symmetric idempotent projector ($M_\mathcal{T}^2 = M_\mathcal{T}$), we have $M_\mathcal{T}\vec{R}_1 \cdot \vec{R}_0 = \vec{R}_1^\top M_\mathcal{T}\vec{R}_0 = \vec{R}_1^\top M_\mathcal{T}^2\vec{R}_0 = M_\mathcal{T}\vec{R}_1 \cdot M_\mathcal{T}\vec{R}_0$.

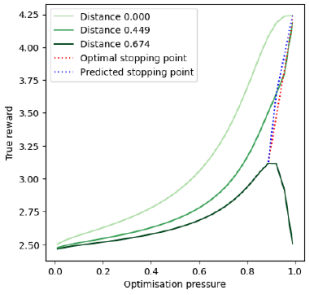


(a) Boundary geometry: the ascent direction (dashed green) is the projection of $M_{\mathcal{T}}\vec{R}_1$ onto the boundary face, deflecting away from $\vec{R}_0$.

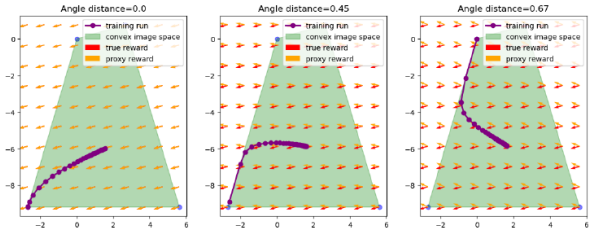
(b) Reward consequence: proxy R_1 continues increasing while true reward R_0 declines after the boundary hit.

Figure 3: Phase 2 (boundary): when steepest ascent hits a face of Ω, the constrained ascent direction can oppose $\vec{R}_0$, causing Goodharting. (From Karwowski et al. [8].)

REPEATED BOUNDARY HITS. As steepest ascent continues, $\vec{\eta}$ encounters successive faces, and each boundary hit further constrains the ascent direction. By the concavity property (5), the angle between $\vec{t}_i$ and the boundary normal *decreases* monotonically — meaning Goodharting becomes *progressively more likely* with increasing optimisation pressure. This explains the characteristic shape of the Goodhart curve: initial improvement is robust, but continued optimisation becomes increasingly dangerous (Figure 4).



(a) Goodharting behavior in $\mathcal{M}_{2,2}$ over three reward functions. Our method is able to predict the optimal stopping time (in blue).



(b) Training runs for each of the reward functions embedded in the state-action occupancy measure space. Even though the full frequency space is $|S||A| = 4$-dimensional, the image of the policy space occupies only a $|S|(|A|-1) = 2$-dimensional linear subspace. Goodharting occurs when the cosine distance between rewards passes the critical threshold and the policy snaps to a different endpoint.

Figure 3: Visualisation of Goodhart's law in case of $\mathcal{M}_{2,2}$.

Figure 4: Left: Goodhart curves for $\mathcal{M}_{2,2}$ at three proxy distances, with predicted optimal stopping points. Right: Occupancy-space trajectories showing how the policy snaps to different vertices as the projected angle crosses the critical threshold. (From Karwowski et al. [8].)

SUMMARY. Hence Goodharting occurs because

1. **Reward misalignment**: larger $\angle(\vec{R}_0, \vec{R}_1)$ makes Goodharting more likely — the further the proxy is from truth, the less margin there is before boundary deflections cause harm.
2. **Optimisation pressure**: by concavity (5), the ascent direction becomes progressively less aligned with the boundary normals, so Goodharting risk increases monotonically with optimisation pressure λ
3. **Polytope geometry**: the shape of Ω (determined by $\mathcal{T}$ and γ) controls where and how sharply the path deflects at each boundary face.

3.1.4 Preventing Goodharting

If the proxy reward is gaining very little per step but the policy is moving a lot in occupancy space, the optimiser is exploiting proxy-specific structure rather than making genuine progress in obtaining higher true rewards. Given a bound $\angle(\vec{R}_0, \vec{R}_1) \leq \theta$, we can make this precise:

Theorem 3 (Optimal stopping)**.** *There exists $\vec{R}_0$ with $\angle(\vec{R}_0, \vec{R}_1) \leq \theta$ that prefers π_A over π_B iff:*

$$\underbrace{\frac{J_{R_1}(\pi_B) - J_{R_1}(\pi_A)}{\|\vec{\eta}^{\pi_B} - \vec{\eta}^{\pi_A}\|}}_{\text{proxy gain per unit movement}} < \underbrace{\sin(\theta)\,\|M_{\mathcal{T}}\vec{R}_1\|}_{\text{threshold from uncertainty}} \tag{6}$$

When the proxy gain per unit of occupancy movement drops below the threshold, some true reward in the uncertainty cone would prefer the earlier policy — stop optimising. For concave optimisation (which holds by (5)), the first violation is worst-case optimal. Across 30,400 configurations, early stopping always achieves NDH = 0 (Goodharting eliminated), at a cost of 10–44% true reward. Smaller θ (tighter bound on misalignment) reduces this cost.

3.1.5 Goodhart's Law in RLHF

The geometric account characterises Goodharting in small MDPs with known structure. Gao et al. [9] provide the first quantitative characterisation of the same phenomenon in RLHF, fitting functional forms to the Goodhart curves that emerge as a policy is optimised against a proxy reward model.

SETUP. A frozen 6B-parameter reward model (from InstructGPT) serves as a "gold" oracle playing the role of the true human preference function. Proxy RMs of varying size (3M to 3B parameters) are trained on synthetic comparisons labelled by the gold RM. Policies (1.2B or 6B SFT models) are then optimised against the proxy. The proxy–gold gap is measured as a function of the KL distance from the initial policy:

$$d = \sqrt{D_{\mathrm{KL}}(\pi \| \pi_{\mathrm{init}})}$$

BEST-OF-n SCALING. In BoN, n outputs are sampled from π_{init} and the highest-scoring one (under the proxy) is selected. The gold RM score follows a concave quadratic:

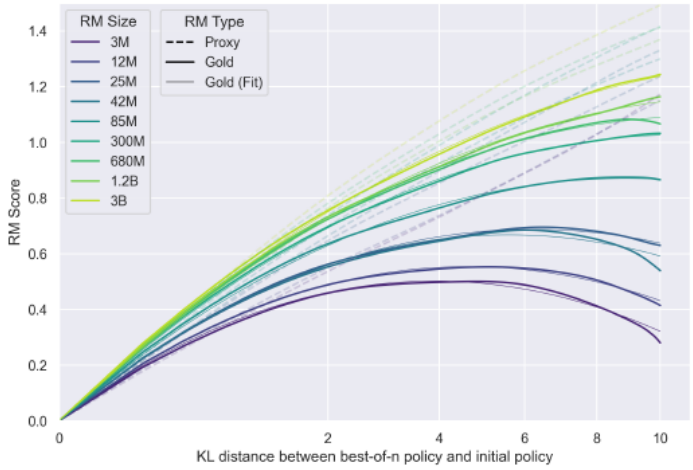
$$R_{\mathrm{BoN}}(d) = d\,(\alpha_{\mathrm{BoN}} - \beta_{\mathrm{BoN}}\,d)$$

peaking at $d^* = \alpha_{\mathrm{BoN}}/(2\beta_{\mathrm{BoN}})$. The linear coefficient α captures genuine improvement (regressional Goodhart — optimisation splits between true reward and noise in proportion to their variances). The quadratic penalty β captures extremal Goodhart — at high optimisation pressure, the proxy's predictions diverge from truth in the tails.

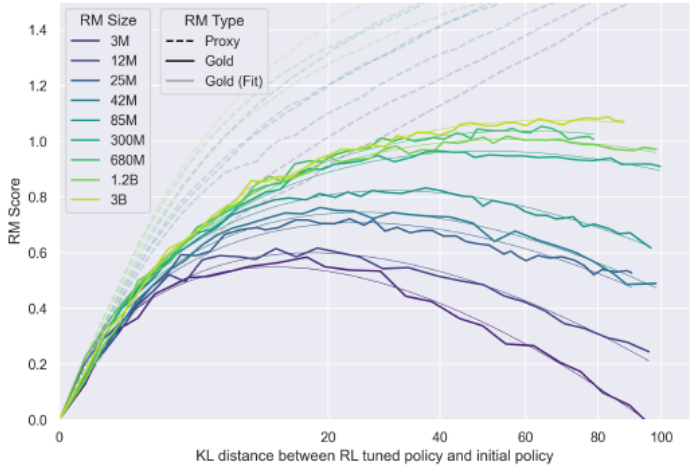
RL (PPO) SCALING. Under PPO optimisation, the gold score follows a different functional form:

$$R_{\mathrm{RL}}(d) = d\,(\alpha_{\mathrm{RL}} - \beta_{\mathrm{RL}}\,\log d)$$

peaking at $d^* = e^{\alpha_{\mathrm{RL}}/\beta_{\mathrm{RL}}-1}$. The logarithmic rather than linear penalty reflects that RL accumulates KL incrementally across the optimisation trajectory, producing more gradual degradation than BoN's one-shot extremal selection. RL is 10–100$\times$ less KL-efficient than BoN for equivalent gold reward gain.



(a) BoN



(b) RL

Figure 1: Reward model (RM) parameter size scaling experiments using the InstructGPT environment. Policy size is held constant (1.2B), while reward model size is varied. The x-axes have a square-root scale. Note that the plots have different x-axes. The gold reward represents the ground truth reward; we observe that when we optimize for a learned proxy of the gold reward, the gold reward initially increases and later decreases. We show that our functional forms fit this effect well.

Figure 5: Gold RM score vs. $\sqrt{D_{\mathrm{KL}}}$ for BoN (top) and RL (bottom) across proxy RM sizes, with fitted scaling laws. (From Gao et al. [9].)

SCALING WITH RM SIZE. α_{RL} is approximately *constant* across RM sizes (3M–3B), while β_{RL} *decreases* log-linearly — larger RMs are more robust to extremal exploitation. The α term captures *regressional* Goodhart (optimisation splits between signal and noise), while the β term captures *extremal* Goodhart (OOD breakdown of the proxy). As we discuss in Section 6, β should be predictable from the reward equivalence class diameter δ.

3.2 Why KL Regularisation Fails

The standard defence against overoptimisation is KL regularisation: optimise $\mathbb{E}[R_\phi(\pi)] - \beta\, D_{\mathrm{KL}}(\pi\|\pi_0)$, constraining the policy to stay close to a reference. Three independent lines of work show this is insufficient.

3.2.1 *Heavy-tailed Errors*

Kwa et al. [10] decompose the proxy reward as $U = X + V$ (misspecification error X + true utility V) and prove that the tail behaviour of X determines whether KL regularisation is safe:

Theorem 4 (Heavy-tailed ⇒ KL fails)**.** *For any heavy-tailed reference distribution Q with mean μ_Q and any $M, \varepsilon > 0$, there exists a distribution P with $\mu_P > M$ and $D_{KL}(P\|Q) < \varepsilon$.*

One can achieve arbitrarily high mean with negligible KL cost by concentrating mass on the heavy tail of Q — the KL penalty for upweighting a rare event at value t scales as $\alpha \log(\alpha/q(t))$, which grows sublinearly in t for heavy-tailed Q. Conversely, if Q is light-tailed and $D_{\mathrm{KL}}(P\|Q) \le d$, then μ_P is bounded and $\mu_P \to \mu_Q$ as $d \to 0$. When error X is heavy-tailed and true utility V is light-tailed, KL-regularised optimisation can push proxy reward to infinity while $\mathbb{E}[V(\pi)] \approx \mathbb{E}[V(\pi_0)]$ — the policy achieves nothing real.

3.2.2 KL does not control distribution shift

Laidlaw et al. [11] identify a different failure mode. Standard RLHF regularisation penalises how much the policy's *action distribution* at each state diverges from the reference. But a proxy $\tilde{R}$ is only reliable on the state-action pairs the reference policy actually visits — once the optimised policy drifts to different states, the proxy–true correlation breaks down entirely. The core issue: small per-step action divergences can compound through the dynamics into drastically different *state visitations*, and action-distribution penalties are blind to this cascading effect.

AD VS. OM REGULARISATION. Define the state-action occupancy measure

$$\mu_\pi(s,a) = (1-\gamma)\,\eta^\pi(s,a)$$

where η^π is the occupancy measure from (1).[9] Standard RLHF uses **action-distribution (AD) regularisation**:

$$\max_\pi \; J_{\tilde{R}}(\pi) \;-\; \lambda\,(1-\gamma)\,\mathbb{E}_\pi\left[\sum_{t=0}^{\infty}\gamma^t\, D_{\mathrm{KL}}\big(\pi(\cdot|s_t)\;\|\;\pi_{\mathrm{ref}}(\cdot|s_t)\big)\right]$$

This penalises how different the *conditional action distributions* $\pi(\cdot|s)$ are from $\pi_{\mathrm{ref}}(\cdot|s)$, averaged over states that π visits. The alternative is **occupancy-measure (OM) regularisation**:

$$\max_\pi \; J_{\tilde{R}}(\pi) \;-\; \lambda\, D\big(\mu_\pi \;\|\; \mu_{\pi_{\mathrm{ref}}}\big)$$

which penalises the divergence between the *joint* state-action distributions directly. The critical difference: AD regularisation can be small even when the policy visits entirely different states from the reference. OM regularisation captures both action changes and their downstream consequences on state visits.

Figure 3: Unlike RLHF, which attempts to prevent reward hacking by regularizing *action distribution* (AD) divergence from the reference policy, our results suggest regularizing using *occupancy measure* (OM) divergence is more effective. These plots of the glucose monitoring environment show the typical ADs and OMs of two policies. π is close to π_{ref} in AD; it gives slightly less insulin. However, π's optimization of the proxy also leads to a vastly different OM with typical glucose levels far outside the healthy range (dotted lines). Thus, regularizing ADs to be close to π_{ref} is not enough to prevent reward hacking. Instead, divergence between the OMs better captures the reward hacking behavior.

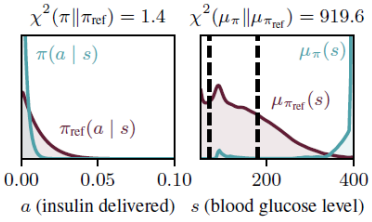


Figure 6: AD vs. OM divergence gap: the optimised policy has small AD χ^2 (actions look similar) but enormous OM χ^2 (state visitations have shifted drastically), demonstrating that AD regularisation is blind to cascading state drift. (From Laidlaw et al. [11].)

Note that at $\gamma = 0$ (contextual bandits), OM = AD, which is why the standard KL penalty works for single-turn RLHF. For sequential or agentic tasks ($\gamma > 0$), the two diverge.

CORRELATED PROXIES. The proxy $\tilde{R}$ is **r-correlated** with the true reward R if their Pearson correlation under $\mu_{\pi_{\mathrm{ref}}}$ equals $r > 0$:

$$\mathbb{E}_{\mu_{\pi_{\mathrm{ref}}}}\left[\frac{\tilde{R} - J_{\tilde{R}}(\pi_{\mathrm{ref}})}{\sigma_{\tilde{R}}} \cdot \frac{R - J_R(\pi_{\mathrm{ref}})}{\sigma_R}\right] = r$$

where $\sigma^2_{\tilde{R}} = \mathrm{Var}_{\mu_{\pi_{\mathrm{ref}}}}(\tilde{R})$ and $\sigma^2_R = \mathrm{Var}_{\mu_{\pi_{\mathrm{ref}}}}(R)$. The correlation is measured under the *reference policy's* occupancy measure — proxy and true rewards need only agree on state-action pairs the designer had in mind, and can diverge arbitrarily elsewhere.

9 The $(1-\gamma)$ factor normalises μ_π to a proper distribution ($\sum_{s,a}\mu_\pi = 1$), so that $J_R(\pi) = \frac{1}{1-\gamma}\,\mathbb{E}_{\mu_\pi}[R]$.Normalisation is needed for divergences $D(\mu_\pi\|\mu_{\pi_{\mathrm{ref}}})$ to be well-defined.

THE TRUE REWARD LOWER BOUND. The central result bounds true reward improvement in terms of proxy improvement and occupancy-measure divergence:

Theorem 5 (True reward lower bound [11]). *For any r-correlated proxy $\tilde{R}$ and any policy π with $\mu_\pi \ll \mu_{\pi_{ref}}$:*

$$\frac{J_R(\pi) - J_R(\pi_{ref})}{\sigma_R} \geq \frac{1}{r}\left(\frac{J_{\tilde{R}}(\pi) - J_{\tilde{R}}(\pi_{ref})}{\sigma_{\tilde{R}}} - \sqrt{(1-r^2)\,\chi^2(\mu_\pi \| \mu_{\pi_{ref}})}\right) \tag{7}$$

where $\chi^2(\mu_\pi \| \mu_{\pi_{ref}}) = \sum_{s,a} \frac{(\mu_\pi(s,a) - \mu_{\pi_{ref}}(s,a))^2}{\mu_{\pi_{ref}}(s,a)}$.

The bound has three terms with a clear interpretation:

- **Proxy gain** $\Delta_{\tilde{R}} = (J_{\tilde{R}}(\pi) - J_{\tilde{R}}(\pi_{\text{ref}}))/\sigma_{\tilde{R}}$: the normalised gain in proxy reward.
- **Distribution shift penalty** $\sqrt{(1-r^2)\,\chi^2(\mu_\pi \| \mu_{\pi_{\text{ref}}})}$: the cost of moving the occupancy measure away from the reference. Larger shift $\Rightarrow$ more opportunity for proxy–true decorrelation.
- **Correlation discount** $1/r$: weaker proxy–true correlation amplifies the penalty.

True reward improvement is guaranteed positive whenever the proxy gain exceeds the distribution-shift penalty. This directly motivates the regularised objective:

$$\max_\pi \; J_{\tilde{R}}(\pi) - \lambda\sqrt{\chi^2(\mu_\pi \| \mu_{\pi_{\text{ref}}})}, \qquad \lambda = \sigma_{\tilde{R}}\sqrt{1-r^2}$$

The regularisation strength λ *decreases* as correlation r increases: a better proxy needs less regularisation.

WHY χ^2 OVER KL. Both divergences penalise deviation from the reference, but they differ critically in how they handle large excursions. Writing both as expectations of a function g of the log-ratio $d(s,a) = \log(\mu_\pi(s,a)/\mu_{\pi_{\text{ref}}}(s,a))$:

$$g_{\text{KL}}(d) = d + e^{-d} \quad \text{grows } \textbf{linearly} \text{ as } d \to \infty$$
$$g_{\chi^2}(d) = e^d + e^{-d} \quad \text{grows } \textbf{exponentially} \text{ as } d \to \infty$$

When $\mu_\pi(s,a) \gg \mu_{\pi_{\text{ref}}}(s,a)$ (the policy visits some state-action pair far more than the reference), KL barely notices while χ^2 penalises it exponentially.

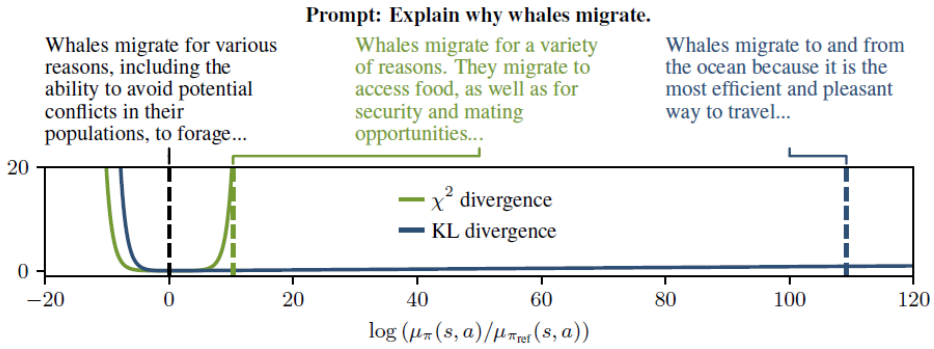


Figure 4: Our theory also suggests reward hacking can more effectively be prevented by regularizing with χ^2 *divergence* instead of *KL divergence*. This plot illustrates how χ^2 regularization is more effective at preventing reward hacking in RLHF. Both divergences can be written as the expectation of a function $g\big(\log\left(\mu_\pi(s,a)/\mu_{\pi_{\text{ref}}}(s,a)\right)\big)$ which increases the penalty on state-action pairs based on how far the log-ratio is from zero. The $g(\cdot)$ associated with KL divergence only increases slowly for large log-ratios, so policies trained with KL divergence may produce nonsensical text. In contrast, the $g(\cdot)$ for χ^2 divergence increases exponentially, better constraining the LLM to produce text similar to the SFT policy.

Figure 7: g_{KL} grows linearly while g_{χ^2} grows exponentially in the log density ratio, explaining why KL regularisation under-penalises large occupancy-measure excursions. (From Laidlaw et al. [11].)

AD REGULARISATION PROVABLY FAILS. A separate negative result shows the failure of *any* action-distribution regularisation in sequential settings:

Theorem 6 (AD regularisation is insufficient [11]). *For any $r \in (0,1)$, any f-divergence[10] D_f, and any monotone scaling g, there exists an MDP with an r-correlated proxy $\tilde{R}$ and a policy $\tilde{\pi}$ such that*

$$J_{\tilde{R}}(\tilde{\pi}) - J_{\tilde{R}}(\pi_{ref}) - g\big(D_f(\pi(\cdot|s)\|\pi_{ref}(\cdot|s))\big) > 0 \quad \textit{but} \quad J_R(\tilde{\pi}) < J_R(\pi_{ref})$$

The policy appears to improve by the AD-regularised criterion, yet true reward *decreases*. This holds universally — no choice of f-divergence or scaling function can fix it.

10 An f-divergence is $D_f(P\|Q) = \mathbb{E}_Q[f(P/Q)]$ for a convex function f with $f(1) = 0$. This family includes KL divergence ($f(t) = t\log t$), χ^2 divergence ($f(t) = (t-1)^2$), total variation ($f(t) = |t-1|/2$) etc.

THE STATISTICAL PERSPECTIVE. Huang et al. [12] arrive at the same conclusion from sample complexity. The **concentrability coefficient** $C^\pi = 1 + D_{\chi^2}(\pi \| \pi_{\text{ref}})$ controls how reward estimation error amplifies when transferring from π_{ref} to π:

$$\mathbb{E}_\pi\big[(\hat{R} - R^*)^2\big] \;\le\; C^\pi \cdot \varepsilon_{\text{stat}}$$

Under KL-regularisation, optimal density ratios scale as $\pi^*_{\text{KL}}/\pi_{\text{ref}} \lesssim \exp(R_{\max}/\beta)$ — *exponential* in the reward range, making C^π exponentially large. Under χ^2-regularisation, ratios become $\pi^*_{\chi^2}/\pi_{\text{ref}} \lesssim 1 + R_{\max}/\beta$ — *linear*. This clips the heavy tail, preventing concentration on uncovered actions. No β resolves the KL dilemma: small β permits extreme overoptimisation; large β keeps the policy uselessly close to π_{ref}.

3.3 Defences and Their Limits

RLHF VS DPO: MIS-SPECIFICATION DETERMINES WHICH OVEROPTIMISES. Shi et al. [13] show that DPO implicitly learns a surrogate reward $\hat{R}_\theta = \beta \log(\pi_\theta/\pi_{\text{ref}})$ in a different function class $\mathcal{F}_\Pi$ than RLHF's explicit $\mathcal{F}$. Which method overoptimises less depends entirely on which class is mis-specified:

- **Policy mis-specification** ($R^* \in \mathcal{F}$, $\pi^* \notin \Pi$): RLHF recovers the exact reward, then finds the best policy in Π; DPO can be strictly sub-optimal because $\mathcal{F}_\Pi$ is constrained by the policy parameterisation.
- **Reward mis-specification** ($R^* \notin \mathcal{F}$, $\pi^* \in \Pi$): DPO bypasses lossy reward modelling and directly recovers π^*; RLHF optimises a wrong objective.

Under finite samples, the separation sharpens: if the true reward is k-sparse in d features, RLHF achieves $\tilde{O}(\sqrt{k \log d / n})$ error while DPO suffers $\Omega(d/n)$ — rewards are simpler to learn than Q-functions, giving RLHF a structural advantage when data is scarce.

4 NEUROSCIENCE OF GOAL-DIRECTED BEHAVIOUR

The neuroscience of goal-directed behaviour provides an empirically grounded framework for understanding how agents acquire, represent, and misrepresent goals. Its central distinction — between **goal-directed** and **habitual** control [14, 15] — closely relates to goal misgeneralization. A **model-based** (goal-directed) controller learns the transition function $\tilde{\mathcal{T}}$ and reward function $\tilde{r}$, then evaluates actions by searching a decision tree; because outcomes are explicitly represented, changing the value of an outcome immediately changes the policy. A **model-free** (habitual) controller stores cached action values $Q(s,a)$ updated by TD errors — scalar summaries divorced from the identity of the outcomes they predict.

Revaluation experiments [16] operationalise this distinction. Three types target different components:

- **Reward revaluation**: change $R(s')$ at a terminal state, hold $\mathcal{T}$ fixed. Tests whether a local reward change propagates to distal values without re-experiencing connecting trajectories.
- **Transition revaluation**: change $\mathcal{T}(s' \mid s,a)$ at one state (e.g. block a passageway), hold R fixed. Tests whether the agent re-routes from a distal start given only local experience of the changed dynamics.
- **Policy revaluation**: hold $\mathcal{T}$ and existing R fixed, introduce a new reward $R_2 > R_1$ at a location the trained policy never visits. Tests whether the agent switches to the superior goal — i.e. whether it discovers that the optimal policy has changed, despite its learned dynamics T^π assigning near-zero occupancy to the new reward's region.

	What changes	MB	MF	SR-TD
Reward reval.	$R(s')$ at terminal	✓	×	✓ ($V = M^\pi R^\pi$)
Transition reval.	$\mathcal{T}(s'\|s,a)$ at one state	✓	×	× (M^π compiled, stale)
Policy reval.	New $R_2 > R_1$ off-policy	✓	×	× (T^π policy-dependent)

4.1 The Successor Representation

The **successor representation** (SR), introduced by Dayan [17], occupies an intermediate position between MF and MB. It caches not the scalar value itself, but a *predictive map* of future states — a multi-step summary of "where the agent expects to be" that can be rapidly combined with reward information to produce values. This factorisation confers some of the flexibility of MB systems (immediate adaptation to reward changes) while retaining the computational efficiency of MF systems (value computation is a single dot product rather than an iterative tree search).

DERIVATION. Dayan [17] observed that $V_\pi(s)$ can be decomposed linearly into a matrix representing how often policies will visit states, and the expected rewards of states. Define the SR M^π as a matrix whose (s, s') entry gives the expected discounted future occupancy of state s' along trajectories starting from s under policy π:

$$M^\pi(s, s') = [M^\pi]_{s,s'} = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t \mathbb{1}[s_t = s'] \;\middle|\; s_0 = s\right] \tag{8}$$

where $\mathbb{1}[s_t = s'] = 1$ if the agent is in state s' at time t and o otherwise. The entry $M^\pi(s, s')$ counts the expected number of (discounted) visits to s' if the agent starts in s and follows π. Nearby states that appear frequently along trajectories from s have high entries; distant or rarely-visited states have entries near zero. The value function is then recovered as a dot product of the SR with the policy-weighted one-step reward $R^\pi(s) = \sum_a \pi(a \mid s)\, R(s, a)$. To see why, note that the expected immediate reward at any state s under π is $\mathbb{E}_\pi[r_{t+1} \mid s_t = s] = R^\pi(s)$, then expand:

$$\begin{aligned} V_\pi(s) &= \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t\, r_{t+1} \;\middle|\; s_0 = s\right] = \sum_{t=0}^{\infty} \gamma^t\, \mathbb{E}_\pi[r_{t+1} \mid s_0 = s] \\ &= \sum_{t=0}^{\infty} \gamma^t \sum_{s'} \Pr(s_t = s' \mid s_0 = s,\, \pi)\; \mathbb{E}_\pi[r_{t+1} \mid s_t = s'] \\ &= \sum_{s'} \left[\sum_{t=0}^{\infty} \gamma^t\, \Pr(s_t = s' \mid s_0 = s,\, \pi)\right] R^\pi(s') \\ &= \sum_{s'} [M^\pi]_{s,s'}\, R^\pi(s') \end{aligned} \tag{9}$$

In vector form, writing $\vec{V}_\pi, \vec{R}^\pi \in \mathbb{R}^{|\mathcal{S}|}$:

$$\vec{V}_\pi = M^\pi\, \vec{R}^\pi$$

The expected return from an initial state distribution $\vec{\mu}$ is $J(\pi) = \vec{\mu}^\top M^\pi \vec{R}^\pi$.[11]

CLOSED FORM. Let $T^\pi \in \mathbb{R}^{|\mathcal{S}| \times |\mathcal{S}|}$ be the state-to-state transition matrix induced by π,:

$$[T^\pi]_{s,s'} = \sum_a \pi(a \mid s)\, \mathcal{T}(s' \mid s, a)$$

The n-step transition matrix $(T^\pi)^n$ satisfies:

$$[(T^\pi)^n]_{s,s'} = \Pr(s_n = s' \mid s_0 = s,\, \pi)$$

so $[M^\pi]_{s,s'} = \sum_{n=0}^{\infty} \gamma^n\, [(T^\pi)^n]_{s,s'}$. Let $\mathbf{1}_s$ be the one-hot indicator vector for state s. In row form:

$$[M^\pi]_{s,:} = \mathbf{1}_s + \gamma\, [T^\pi]_{s,:} + \gamma^2\, [(T^\pi)^2]_{s,:} + \cdots = \sum_{n=0}^{\infty} \gamma^n\, [(T^\pi)^n]_{s,:}$$

Recognising the Neumann series $\sum_{n=0}^{\infty} (\gamma T^\pi)^n = (I - \gamma T^\pi)^{-1}$:

$$M^\pi = \sum_{n=0}^{\infty} \gamma^n\, (T^\pi)^n = (I - \gamma\, T^\pi)^{-1} \tag{10}$$

M^π encodes the *entire* multi-step transition structure of the MDP under π. [12]

LEARNING THE SR VIA TD. Just as V_π satisfies a Bellman recursion (Eq. **??**), so does each entry of M^π. For a fixed target state s', the entry $M^\pi(s, s')$ satisfies:[13]

$$M^\pi(s, s') = \mathbb{1}[s = s'] + \gamma \sum_{s''} [T^\pi]_{s,s''}\, M^\pi(s'', s')$$

where the sum is over *intermediate* states s'' that s can transition to. This has the same structure as the Bellman equation for $V_\pi(s)$: the update is $\mathbb{1}[s = s']$ ("am I at target s' right now?") and the "next-state value" is $M^\pi(s'', s')$ ("starting from s'', how often will I visit s' in the future?"). After observing a transition from $s \to s''$, we can use the same TD updates to update each target s',

$$\begin{aligned} \delta(s') &= \underbrace{\mathbb{1}[s = s'] + \gamma\, M^\pi(s'', s')}_{\text{target}} - \underbrace{M^\pi(s, s')}_{\text{current estimate}} \\ M^\pi(s, s') &\leftarrow M^\pi(s, s') + \alpha\, \delta(s') \end{aligned} \tag{11}$$

11 This generalises to state-action pairs via the **occupancy measure** $\vec{\eta}^\pi \in \mathbb{R}^{|\mathcal{S}||\mathcal{A}|}$, with $[\vec{\eta}^\pi]_{s,a} = [\vec{\mu}^\top M^\pi]_s \cdot \pi(a \mid s)$, so that $J(\pi) = \vec{\eta}^\pi \cdot \vec{R}$. The state marginal $\sum_a [\vec{\eta}^\pi]_{s,a} = [\vec{\mu}^\top M^\pi]_s$ is the SR row weighted by μ; for a single starting state this is $M^\pi(s, s')$, i.e. the Need term in the EVB framework (Section 4.2). The set of all achievable $\vec{\eta}^\pi$ forms a convex polytope whose geometry governs Goodhart's law (Section 3).

12 Substituting (10) into (9) gives $\vec{V}_\pi = M^\pi\, \vec{R}^\pi = (I - \gamma T^\pi)^{-1}\, \vec{R}^\pi$, which is equivalent to the Bellman equation $\vec{V}_\pi = \vec{R}^\pi + \gamma\, T^\pi \vec{V}_\pi$ solved for $\vec{V}_\pi$ — confirming that the SR decomposition is exact.

13 In matrix form this is $M^\pi = I + \gamma\, T^\pi M^\pi$, or equivalently in row-vector form $[M^\pi]_{s,:} = \mathbf{1}_s + \gamma \sum_{s''} [T^\pi]_{s,s''}\, [M^\pi]_{s'',:}$.

TEMPORAL STRUCTURE. This SR factorization allows the weights to update quickly to reward changes. The value function is approximated as $\hat{V}(s) = \sum_{s'} [M^\pi]_{s,s'} w(s')$, where w is a weight vector learned by TD. Comparing with Eq. (9), the optimal weights are simply $w^*(s) = R^\pi(s)$ — the policy-weighted immediate reward at each state. This means the weight vector w contains *no temporal information whatsoever*: all the multi-step dynamics are absorbed into M^π, and w is a purely local quantity that can be learned from single transitions without chaining or bootstrapping. This is why the SR enables fast adaptation to reward changes: updating $R^\pi(s)$ at a single state immediately propagates to the values of all states whose SR includes s.

THE SR ALGORITHM FAMILY. Russek et al. [16] explored three strategies for computing M^π:

- **SR-TD**: cache M^π and update via TD (11). Passes reward revaluation (changing R immediately propagates via $V = M^\pi R^\pi$) but fails transition revaluation (changing $\mathcal{T}$ requires re-experiencing affected trajectories to update M^π).
- **SR-MB**: learn T^π and recompute $M^\pi = (I - \gamma T^\pi)^{-1}$ at decision time. Passes both reward and transition revaluation, but fails **policy revaluation**: since T^π depends on π learned from past behaviour, a new reward that should change the policy is not fed back into the SR computation.
- **SR-Dyna**: store experienced transitions (s, a, s') and replay k of them offline between decisions, updating a state-action SR matrix H with off-policy updates (using the greedy action under current rewards rather than the action actually taken). With large k, passes all three revaluation types; with small k, degrades gracefully to SR-TD.

	Reward reval.	**Trans. reval.**	**Policy reval.**	**Decision cost**
MF TD	×	×	×	Lowest
SR-TD	✓	×	×	Low
SR-MB	✓	✓	×	Medium
SR-Dyna (k large)	✓	✓	✓	Med. (offline)
SR-Dyna (k small)	✓	×	×	Low
Full MB	✓	✓	✓	Highest

An MF learner fails both revaluation types equally; an MB learner passes both; any MB/MF mixture predicts symmetric performance. Only the SR passes reward revaluation but fails transition revaluation. Whether deep RL agents trained under narrow distributions (e.g. CoinRun PPO agents [2], which use standard MF architectures) develop implicit representations that exhibit SR-like characteristics, and how it relates to goal misgeneralisation is an open empirical question that we address in Section 6.

4.2 Planning, Replay, and Prioritised Experience

Which experiences does the brain choose to replay — and therefore which parts of state space does it learn about? The same prioritisation that makes replay efficient may also create blind spots at exactly the states where proxy–true reward divergence would be detectable.

PRIORITISED REPLAY AS BELLMAN BACKUPS. Mattar & Daw [18] develop a normative theory of replay within the Dyna framework (Section **??**). Recall that in Dyna-Q (Algorithm **??**), the agent supplements direct RL with k simulated Bellman backups per real transition, where each backup replays a stored experience $e_k = (s_k, a_k, r_k, s'_k)$ and updates:

$$Q(s_k, a_k) \leftarrow Q(s_k, a_k) + \alpha \left[r_k + \gamma \max_{a'} Q(s'_k, a') - Q(s_k, a_k) \right]$$

Standard Dyna selects which experience to replay *uniformly at random*. Mattar & Daw ask: which experience should a rational agent replay to maximise future cumulative reward $J(\pi)$?

THE EXPECTED VALUE OF BACKUP (EVB). Let the agent be in state $S_t = s$. The **expected value of backup** of replaying experience e_k — updating $Q(s_k, a_k)$ — is defined as the improvement in expected return caused by the resulting policy change:

$$\mathrm{EVB}(s_k, a_k) = J(\pi_{\mathrm{new}}) - J(\pi_{\mathrm{old}})$$

where π_{old} and π_{new} are the policies before and after the backup. Therefore, EVB decomposes into two factors — how much better the new policy is *at* s_k, and how often s_k will be visited from the current state s:

$$\mathrm{EVB}(s_k, a_k) = \underbrace{\mathrm{Gain}(s_k, a_k)}_{\text{policy improvement at } s_k} \times \underbrace{\mathrm{Need}(s_k)}_{\text{expected future visits to } s_k} \tag{12}$$

GAIN. Gain measures the expected improvement in return accrued each time the agent visits s_k, given that the backup changes the policy there:

$$\text{Gain}(s_k, a_k) = \sum_{a \in \mathcal{A}} \left[\pi_{\text{new}}(a \mid s_k) - \pi_{\text{old}}(a \mid s_k)\right] Q_{\pi_{\text{new}}}(s_k, a)$$

NEED. Need measures the discounted expected number of future visits to the target state s_k, starting from the agent's current state:

$$\text{Need}(s_k) = \sum_{i=0}^{\infty} \gamma^i \Pr(S_{t+i} = s_k \mid S_t = s, \pi_{\text{old}})$$

This is precisely the (s, s_k) entry of the **successor representation** $M^{\pi_{\text{old}}}(s, s_k)$ from Eq. (8). States the agent expects to visit imminently have high Need; distant or unlikely states have Need near zero.

GOAL MISGENERALIZATION. The EVB decomposition explains why rational computation allocation reinforces proxy goals. A distinguishing state — one where the proxy and true reward disagree — has near-zero EVB under a narrow training policy:

1. **Low Need**: $\text{Need}(s_k) = M^{\pi}(s, s_k)$ is computed under the training policy, so states off the training trajectory have negligible expected occupancy.
2. **Low Gain**: Q-values at unvisited states have never been updated, so no prediction error exists and the backup produces no policy change.

The product Gain $\times$ Need ≈ 0 means the agent never allocates computation to the states that would expose the proxy–true divergence. The same rational prioritisation that makes replay efficient creates systematic blind spots at exactly the states where misgeneralization could be detected. This formalises why Langosco et al.'s $p = 2\%$ training diversity result (Section 2.1) works: randomised coin placements generate prediction errors at novel states (boosting Gain) and increase M^{π} mass there under the diversified policy (boosting Need), directing computation to the states under rewards in proxy-true reward gap.

5 EMERGENT PLANNING IN RL AGENTS

5.1 Model-Free Planning and the DRC Architecture

Guez et al. [19] demonstrate that a generic recurrent architecture trained end-to-end by model-free RL exhibits planning behaviours in Sokoban environments.

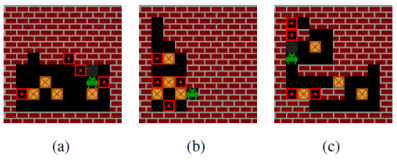


Figure 2. Examples of Sokoban levels from the (a) unfiltered, (b) medium test sets, and from the (c) hard set. Our best model is able to solve all three levels.

Figure 8: Example Sokoban levels from the unfiltered, medium, and hard test sets. The agent (green) must push boxes (brown) onto targets (red crosses). (From Guez et al. [19].)

THE DRC ARCHITECTURE. The **Deep Repeated ConvLSTM** (DRC) is parameterised by a stack depth D (number of ConvLSTM layers) and a repeat count N (number of internal "ticks" per environment step).[14] The basic element is a ConvLSTM — an LSTM whose hidden state is a 3D tensor (preserving the spatial dimensions of the input) and whose linear operations are replaced by convolutions. A recurrent network f_θ stacks D such modules with unshared weights $\theta = (\theta_1, \ldots, \theta_D)$. The state s contains all cell states c_d and hidden states h_d for each module d:

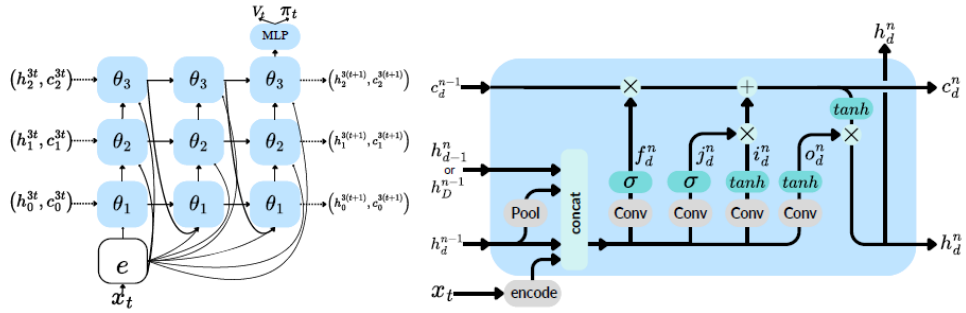
$$s = (c_1, \ldots, c_D,\ h_1, \ldots, h_D)$$

At each environment step, an encoder network e processes the observation x_t into a tensor $i_t = e(x_t)$, which is fed to the recurrent module. The key design choice is the **repeat recurrence**: the stacked ConvLSTM f_θ is applied N times within a single environment step, giving the network N internal computational ticks before it must commit to an action:

$$s_t = g_\theta(s_{t-1}, i_t) = \underbrace{f_\theta(f_\theta(\cdots f_\theta(s_{t-1}, i_t), \cdots, i_t), i_t)}_{N \text{ times}}$$

The output $o_t = h_D$ (the hidden state of the deepest module after N ticks) is flattened and passed through an MLP to produce a policy π_t and value estimate v_t. The cell state c_d^n is the agent's persistent spatial memory; subsequent interpretability work (Section 5.2) reveals that this is where the agent's plans are stored.

14 A "tick" is one pass through the D-layer ConvLSTM stack within a single environment step; N ticks occur before the agent commits to an action. This is distinct from *thinking steps*, which are forced no-op environment steps that give the agent additional batches of N ticks on the same observation.



(a) DRC(3,3) architecture (b) The ConvLSTM block

Figure 2: **Left**: The DRC$(3,3)$ architecture. There are three layers of ConvLSTM modules with all the layers repeatedly applied three times before predicting the next action. **Right**: The ConvLSTM block in the DRC. Note the use of convolutions instead of linear layers, and the last layer of the previous tick (h_D^{n-1}) as input to the first layer. "Pool" refers to a weighted combination of mean- and max-pooling.

Figure 9: The DRC(3,3) architecture: three layers of ConvLSTM modules, each applied N times per environment step. The ConvLSTM block (right) preserves spatial structure via convolutional gates. (From Taufeeque et al. [20].)

Training uses IMPALA (V-trace actor-critic).

PLANNING. Rather than defining planning by its mechanism (explicit model + search), Guez et al. adopt a behavioural definition: an agent plans if its interactions exhibit three properties.

1. **Combinatorial generalisation**: the agent performs well across a combinatorial space of procedurally generated configurations, where memorisation is infeasible.
2. **Data efficiency**: the agent learns effectively from a limited number of unique environments.
3. **Beneficial extra thinking time**: the agent's performance improves when given additional computation *on the same input*, without new observations.

The third criterion is the most distinctive: a purely reactive policy cannot benefit from extra computation on the same input, so improvement implies the agent has learned an iterative internal algorithm.

The DRC is evaluated on Sokoban — a puzzle where the agent pushes boxes onto targets in a 10×10 grid. The combinatorial state space ($\sim 10^{20}$ configurations) makes raw memorisation impossible. DRC(3,3) substantially outperforms all baselines, including architectures with explicit planning inductive biases. *Beneficial thinking time.* Forced no-op actions at episode start (multiple ticks with the same input observation) improve solve rates by $\sim$5% for DRC(3,3).

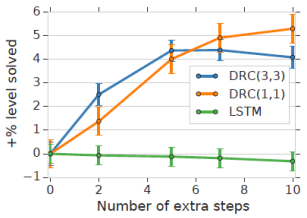


Figure 6. Forcing extra computation steps after training improves the performance of DRC on Sokoban medium-difficulty set (5 networks, each tested on the same 5000 levels). Steps are performed by overriding the policy with no-op actions at the start of an episode. The green line is the LSTM(1,1) model.

Figure 10: Forcing extra no-op computation steps at episode start improves DRC solve rates, confirming beneficial thinking time. (From Guez et al. [19].)

5.2 Interpreting Emergent Plans

Three studies progressively open the DRC's black box. Bush et al. [21] and Taufeeque et al. [22] show plans are linearly decodable, causally active, and refined through adaptive computation allocation. Taufeeque et al. [20] then reverse-engineer the complete planning algorithm at the weight level.

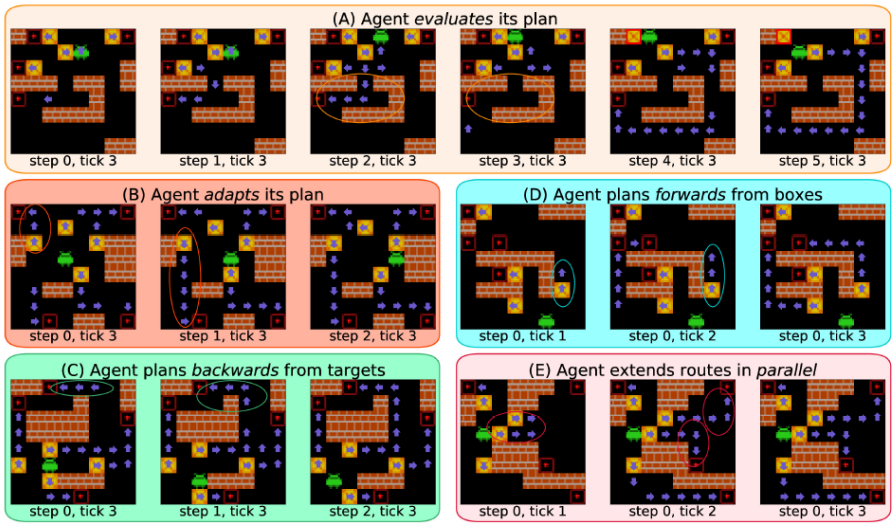


Figure 1: Examples of the DRC agent internally forming plans to push boxes to targets. A purple arrow on a square means that a linear probe decodes that the agent plans to push a box off of that square in the associated direction. No arrow on a square means that the probe decodes that agent does not plan to push a box off of that square. (A) The agent *evaluates* a naively-appealing route, concludes it is infeasible, and forms a longer alternate path. (B) The agent *adapts* its plan and changes the target it plans to push the left-most box to. (C) The agent extends part of its plan *backward* from a target. (D) The agent extends part of its plan *forward* from a box. (E) The agent extends many parts of its plan in *parallel*. We provide further examples in Appendices A.2.1–A.2.5.

Figure 11: Decoded plans from the DRC's cell states: purple arrows show C_B (box push directions) decoded by linear probes. The agent evaluates, adapts, and extends plans across internal ticks — planning forward from boxes and backward from targets in parallel. (From Bush et al. [21].)

5.2.1 *Probing for Representations of Plans*

Bush et al. [21] ask whether the DRC's cell states encode representations of *future* actions — not just the current observation, but the agent's intended plan. They define two planning-relevant concepts, each assigning a 5-class label to every square (x, y) of the 10×10 grid:

- C_A (**Agent Approach Direction**): the direction from which the agent will *next* step onto square (x, y). One of $\{\text{UP}, \text{DOWN}, \text{LEFT}, \text{RIGHT}, \text{NEVER}\}$, where NEVER means the agent will not visit this square again during the episode.
- C_B (**Box Push Direction**): the direction in which a box will *next* be pushed off square (x, y). Same 5-class labelling.

Both concepts are future-predictive: they require observing the full-episode trajectory. Any information about C_A or C_B in the hidden state must reflect the agent's internal plan.

LINEAR PROBES. A linear classifier (logistic regression) with weights $\mathbf{w}_k$ *at each position* (x, y) takes the 32-dimensional internal DRC cell state $\mathbf{g}_{x,y}$ and predicts the class labels k for C_A and C_B.

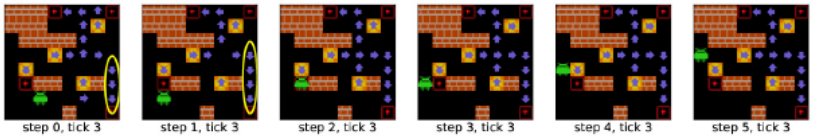
$$l_k = \mathbf{w}_k^\top \mathbf{g}_{x,y}, \qquad \hat{y} = \arg\max_k l_k$$

From Table 1, probes substantially outperform a baseline that has access only to the raw observation (which can recover C_A and C_B labels only for the agent's current position and box positions — everything else requires knowledge of the future). $C_B > C_A$ at every layer: the agent represents box movements more strongly than its own path, reasoning primarily about plans around boxes.

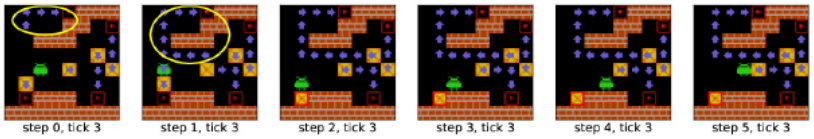
Concept	Layer 1	Layer 2	Observation baseline
C_A (agent approach)	$\sim$0.77	$\sim$0.84	$\sim$0.25
C_B (box push dir.)	$\sim$0.80	$\sim$0.90	$\sim$0.25

Table 1: Linear probe prediction performance (macro F1).

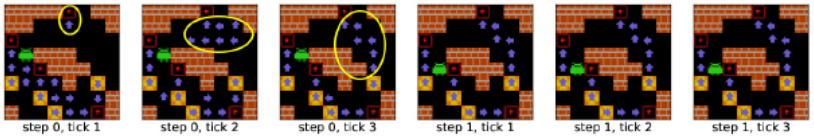
BIDIRECTIONAL PLAN FORMATION. Decoded labels form **continuous chains of arrows**: directed paths from boxes to targets (C_B) and from agent to boxes (C_A). Across internal ticks, forward chains extend from boxes toward targets while backward chains simultaneously extend from targets toward boxes, meeting in the middle. Multiple plan fragments for different boxes extend in parallel. This is emergent **parallelised bidirectional search**.



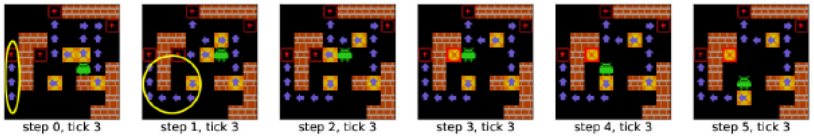
(a) The agent formulates part of its internal plan by iteratively searching backwards from the bottom-right target to the center-most box.



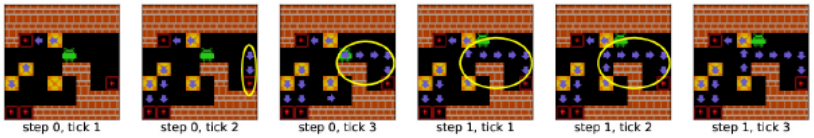
(b) The agent constructs a plan to push a box to the top-most target by iteratively extending a plan backwards from this target. The agent extends this plan backwards until it connects to the top-most box.



(c) The agent forms an internal plan to push a box to the top-most target by iteratively searching backwards from this target. The agent formulates this part of its plan by searching backwards until this plan connects to the lower-right box.



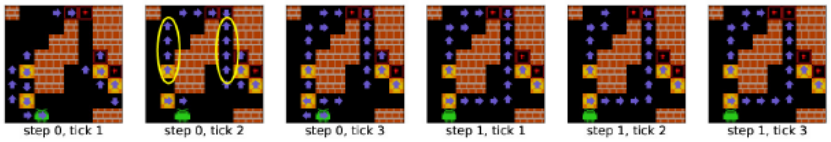
(d) The agent formulates a plan to push the left-most box to the left-most target by searching backwards from the left-most target to the left-most box.



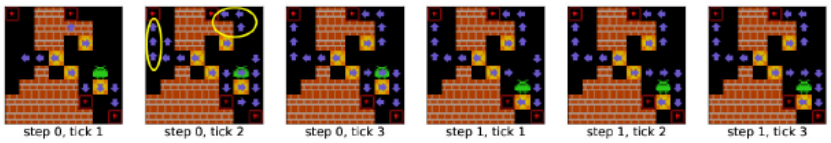
(e) The agent constructs a plan to push a box to the lower-right target by iteratively constructing a plan backwards from this target. The agent extends this plan backwards until it connects to the lower-right box.

Figure 16: Examples of episodes in which the agent formulates its internal plan by iteratively extending planned routes backward from targets. Blue arrows represent the direction that the agent plans to next push a box off of each square. Yellow circles highlight parts of the agent's plan that it has constructed by iteratively searching backwards from targets to boxes. The plans are decoded from the agent's cell state at its first (I16a), second (I16b and I16c) and third (I16d and I16e) layer by a 1x1 probe. The plans are decoded from the agent's cell states at either the final computational tick of the first six steps of episodes (I16a, I16b and I16d), or at each computational tick of the first two steps of episodes (I16c and I16e).

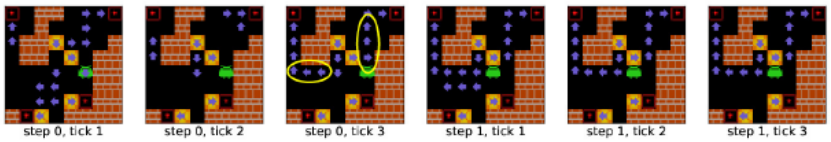
Figure 12: Backward plan formation: the agent simultaneously searches backward from targets toward boxes, extending plans in the reverse direction. (From Bush et al. [21].)



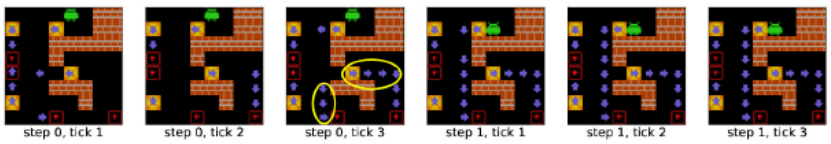
(a) At the fourth computational tick, the agent extends two parts of its plan (e.g. its plans to push boxes to the top-most targets) in parallel.



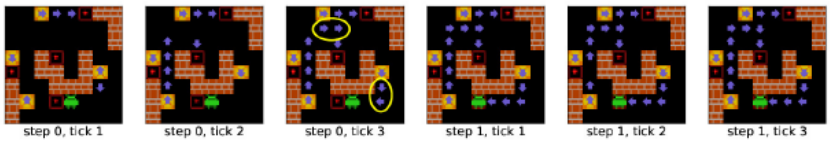
(b) At the second computational tick, the agent iteratively constructs internal plans for the two top-most targets by searching backwards from these two targets in parallel.



(c) At the third computational tick, the agent iteratively extends its internal plans associated with the two top-most targets by extending these two plans in parallel.



(d) The agent searches backwards from the two bottom-most targets in parallel at the third computational tick.



(e) After the third computational tick, the agent extends two parts of its internal plan in parallel.

Figure 17: Examples of episodes in which the agent formulates its internal plan by extending multiple planned routes in parallel over a single computational tick. Blue arrows represent the direction that the agent plans to next push a box off of each square. Yellow circles highlight parts of the agent's plan that it has constructed in parallel over a single tick. The plans are decoded from the agent's cell state at its first (17a), second (17b and 17c) and third (17d and 17e) layer by a 1x1 probe. The plans are decoded from the agent's cell states at either the final computational tick of the first six steps of episodes (17a, 17b and 17d), or at each computational tick of the first two steps of episodes (17c and 17e).

Figure 13: Parallel plan formation: multiple plan fragments for different boxes are constructed simultaneously within a single computational tick. (From Bush et al. [21].)

PLAN REFINEMENT WITH TEST-TIME COMPUTE. During extra thinking steps, the decoded plan improves monotonically: linear probes are a better predictor of future behaviour as performance increases from F1 ∼0.60 to ∼0.85 after 15 additional ticks, plateauing as the chains complete.

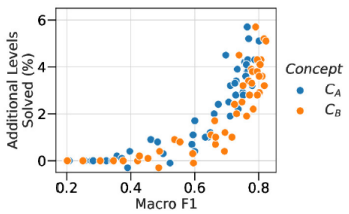


Figure 9: The relationship between the percentage of extra levels, of medium difficulty, solved when an agent is given 5 steps to 'think', and macro F1 score of probes when predicting C_A (blue) and C_B (orange) from the agent's final layer cell state. Each point corresponds to these quantities calculated for a single checkpoint.

Figure 14: Macro F1 of C_A and C_B probes increases with additional no-op ticks, plateauing as plans complete. (From Bush et al. [21].)

CAUSAL INTERVENTIONS. Probing shows that the agent represents plans, but does the plan actually causally drive behaviour? To test this, Bush et al. design handcrafted levels where two solutions exist: a short (optimal) route and a long (suboptimal) route. Without intervention the agent always takes the short route. The intervention modifies the cell state to steer the agent onto the long route via two simultaneous perturbations:

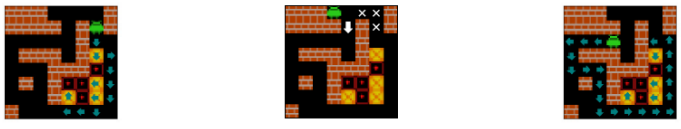
$$\mathbf{g}'_{x,y} \leftarrow \mathbf{g}_{x,y} + \mathbf{w}_k$$

where $\mathbf{w}_k$ is the probe weight vector for target class k. Concretely, (i) the $\mathbf{w}_{\text{NEVER}}$ vector is added to cell state positions along the short path (discouraging the agent from visiting those squares), and (ii) the appropriate directional vector (e.g. $\mathbf{w}_{\text{DOWN}}$) is added at the first square of the long path (encouraging the agent toward it). The short-route intervention is repeated every step; the directional

intervention is applied only until the agent reaches that square. An intervention is **successful** if the agent solves the entire level via the long route — not just a single-step deviation, but a complete multi-step re-plan and execution. In **Agent-Shortcut** levels the agent must walk a longer path to reach the boxes; in **Box-Shortcut** levels a box must be pushed along a longer route to its target. Results across 200 test levels per condition:

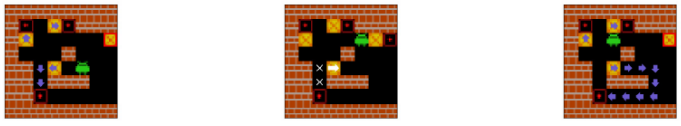
Level type	Layer 1	Layer 2	Layer 3	Random probe
Agent-Shortcut	94.6%	90.1%	**98.8%**	∼30%
Box-Shortcut	56.2%	72.7%	**80.6%**	∼5–30%

Table 2: Causal intervention success rates (% of levels where the agent switches to the intended route) using trained vs. randomly initialised probes.



(a) Plan without intervention (b) Intervention (c) Plan with intervention

Figure 7: An Agent-Shortcut intervention and its effect on the agent's plan as formulated in terms of C_A: (a) the agent's plan after 4 steps *without* the intervention, (b) the initial state of the level and the intervention, and (c) the agent's plan after 4 steps *with* the intervention. The 'short-route' intervention adds the representation of NEVER for C_A to positions with white crosses. The 'directional' intervention adds the representation of DOWN for C_A to the position with the white arrow.



(a) Plan without intervention (b) Intervention (c) Plan with intervention

Figure 8: A Box-Shortcut intervention and its effect on the agent's plan as formulated in terms of C_B: (a) the agent's plan after 4 steps *without* the intervention, (b) the initial state of the level and the intervention, and (c) the agent's plan after 4 steps *with* the intervention. The 'short-route' intervention adds the representation of NEVER for C_B to positions with white crosses. The 'directional' intervention adds the representation of RIGHT for C_B to the position with the white arrow.

Figure 15: Causal interventions on Agent-Shortcut (top) and Box-Shortcut (bottom) levels. Adding probe weight vectors to the cell state steers the agent onto the long route, confirming that plan representations causally drive behaviour. (From Bush et al. [21].)

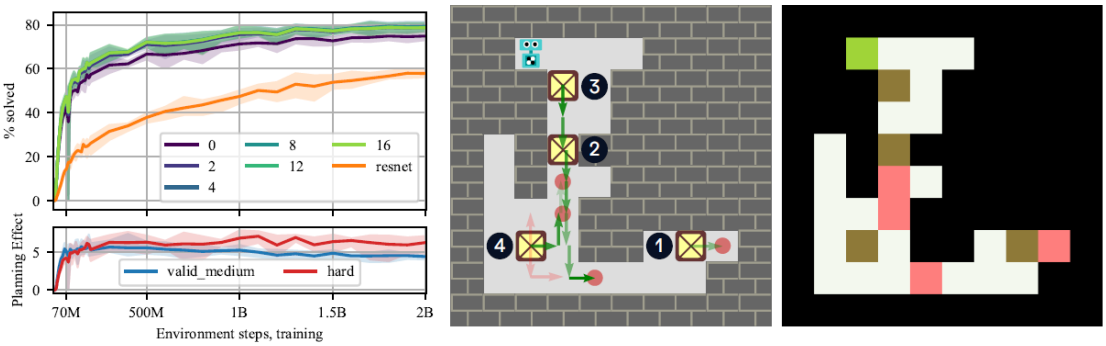


Figure 1: **Top left:** Validation success rate on medium-difficulty levels solved vs. training steps, with varying numbers of initial thinking steps (forced NOOP actions) and a ResNet baseline. **Bottom left:** Estimated planning effect (8-steps minus 0-steps) showing that planning emerges within 70M steps and increases for the hard levels (red) but decreases for medium levels (blue). **Middle:** Linear probe predictions for box movement direction. Green arrows indicate correct predictions and incorrect red arrows indicate plausible alternatives for Box 4. Opacity reflects frequency during the episode. This probe causally affects agent actions as described in Section 3. Boxes are numbered in their order of placement on targets. **Right:** The same level's input representation to the NN, where each pixel is a single tile. Walls are black, boxes are brown, targets are pink, and the robot is green.

Figure 16: Top left: validation success rate vs. training steps with varying numbers of initial thinking steps. Bottom left: estimated planning effect emerges within 70M steps. Middle: linear probe predictions for box movement direction. Right: the level's input representation to the network. (From Taufeeque et al. [22].)

Taufeeque et al. [22] sharpen these results with **adaptive logit thresholding** — the minimal perturbation setting the target-class logit to α:

$$\mathbf{h}' = \mathbf{h} + \hat{\mathbf{p}} \cdot \max\big(0,\ \alpha - \mathbf{h} \cdot \mathbf{p}\big), \qquad \hat{\mathbf{p}} = \frac{\mathbf{p}}{\|\mathbf{p}\|_2^2}$$

which is the closed-form solution to $\arg\min_{\mathbf{h}'} \|\mathbf{h}' - \mathbf{h}\|_2^2$ subject to $\mathbf{h}' \cdot \mathbf{p} \geq \alpha$. With this calibrated intervention, only the Box-Directions probe is strongly causal (F1 = 86.4%, best-case causal effect = 82.5%); all other probes fall below 21%. The agent plans primarily in **box-movement space**, with the probe predicting future box movements with $\geq 90\%$ accuracy up to **50 steps** ahead.

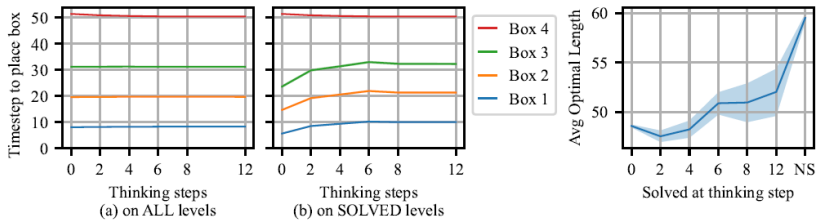


Figure 3: **Left:** Average time step to place each box B_i on target for different numbers of thinking steps. **(a)** Averages across all levels. **(b)** Averages over levels solved by 6 thinking steps but not solved by 0 thinking steps. More thinking steps make the DRC avoid greedy strategies in favor of long-term return. The 95% confidence intervals computed with the bootstrap method over the levels is very small and not visible in the plot. **Right:** Average optimal solution length of levels grouped by the number of thinking steps at which the level is first solved. Levels that take longer to solve tend to be harder. NS stands for "not solved".

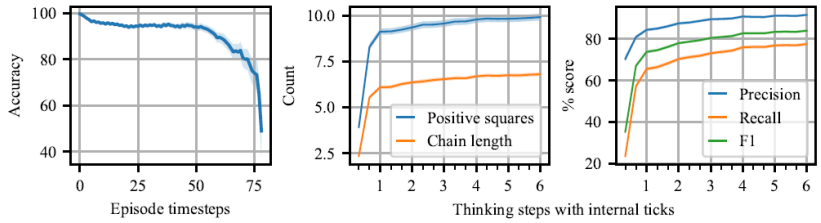


Figure 4: **Left:** The number of steps in advance that the Box-Directions probe is able to predict a box move. The probe predicts $\geq 90\%$ of the actions 50 steps before they occur. **Middle and Right:** The plan quality, as measured by summing length of probe-predicted chain of box-directions starting from boxes and counting squares with positive non-empty predictions, and the F1-score increases over thinking steps (Section 3.1). This suggests that the DRC refines its plan during computation, including across the three ticks per environment step.

Figure 17: Left: thinking steps reduce average time to solve and help with harder levels. Right: probe accuracy and plan chain length increase with internal ticks, confirming the DRC refines its plan during computation. (From Taufeeque et al. [22].)

TEST-TIME COMPUTE. The DRC also exhibits *pacing behaviour*: deliberately walking in a cycle when plans are incomplete (F1 = 51% at cycle onset vs 68% otherwise), self-allocating computation for plan refinement. The evidence is consistent with iterative heuristic refinement rather than classical tree search. Whether the DRC's internal plans target the training reward, or a proxy, is addressed in Section 5.2.2.

5.2.2 *Representation of Plans in Weights*

Taufeeque, Tucker, Gleave & Garriga-Alonso [20] find the agent's plan is reflected **directly in interpretable channel activations** — individual channels of the ConvLSTM hidden state h_d have high activation at position (x, y) if and only if a box or agent will move in that channel's assigned direction from that position at a future timestep. Plans are readable by simply visualising the raw hidden state, no probes needed.

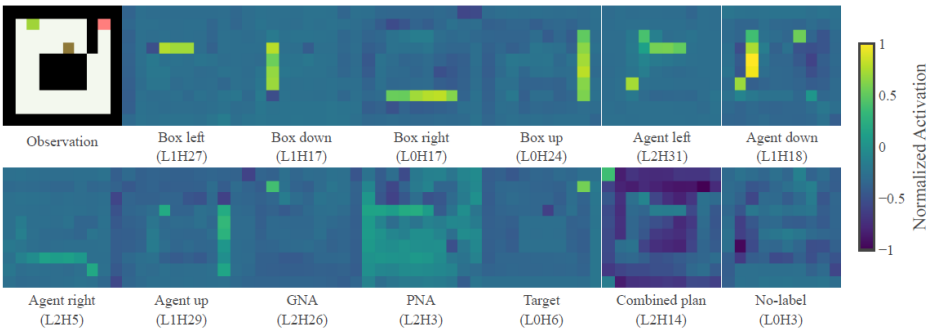


Figure 1: A level with channel activations for a single channel from every group in table 1. Note the clear activations for the box move channels along the box ■'s path to the target ■.

Figure 18: Channel activations for a single level, one channel from each group. Box-movement channels show clear activation along the box's planned route to the target; other channels encode entity positions or next actions. (From Taufeeque et al. [20].)

CHANNEL TAXONOMY. Manual inspection of all 96 channels across 3 layers reveals that **59 of 96** are *path channels* encoding future movement directions (box-movement, agent-movement, and combined-path channels), while the remaining 37 encode entity positions, immediate next actions, or are uninterpretable.

THE PLANNING ALGORITHM IN WEIGHTS. The complete planning algorithm has three mechanistically distinct stages (Figure 19): initialisation from the observation, bidirectional plan extension via recurrent kernels, and winner-takes-all selection between competing plans.

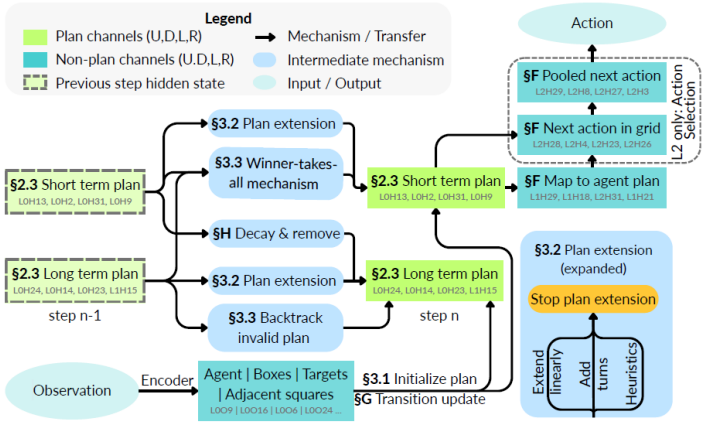


Figure 11: The planning algorithm circuit learned by DRC(3, 3). While the plan nodes are present and updated across all the layers, this circuit only shows the short and long-term plan nodes in the first layer's hidden state (L0HX) with a channel X for each direction up, down, left, and right. Mechanisms are annotated with the sub-section they are studied in section 3.

Figure 19: The full planning circuit learned by DRC(3,3). The encoder seeds short- and long-term plan channels from the observation; plan extension (LPE/TPE) and WTA operate recurrently each tick; backtracking removes invalid plans via negative wall activations. Layer 2 additionally maps plans to action selection. (From Taufeeque et al. [20].)

Stage 1 — Initialisation. The encoder convolutions can be algebraically combined with each ConvLSTM input convolution into a single 9×9 combined kernel mapping directly from the RGB observation to each path channel. These combined kernels reveal clear entity-detection structure (agent, box, target filters) and seed plan chains bidirectionally: **forward chains** initiate at squares adjacent to boxes, extending toward targets; **backward chains** initiate adjacent to targets, extending toward entities.

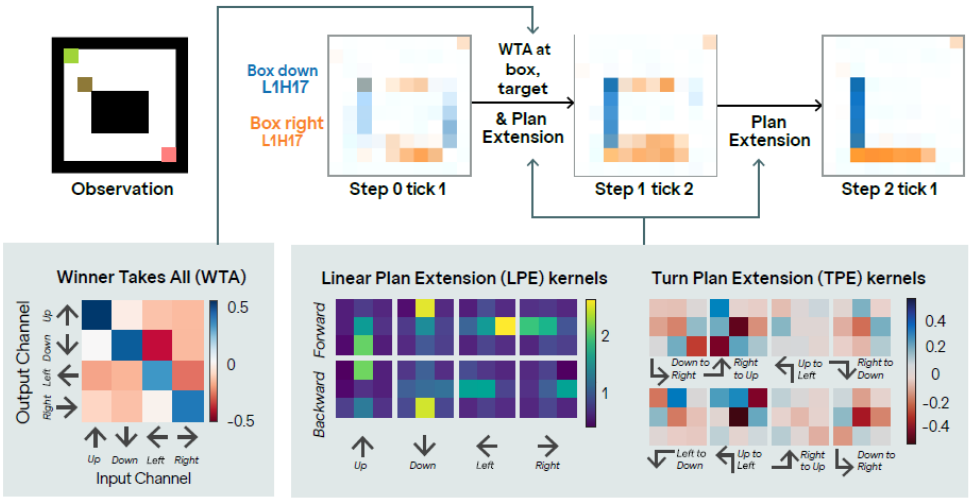


Figure 10: A situation with two equally good paths from the box ■ to the target ■. The sum of box-down (L1H17) and box-right (L1H13) channels shows that the network searches forward from the box and backward from the target. Both paths (down-then-right and right-then-down) are visible at step 0 tick 1 (left) due to the encoder; and the down and right channels have similar activations on the box square (gray). From step 0 tick 1 until step 2 tick 1 (section 2.1 defines 'tick'), the plans are extended in the same direction by Linear Plan Extension (LPE) kernels (bottom-middle) and extended into switching directions by Turn Plan Extension kernels (bottom-right), stopping (fig. 8) on signals corresponding to reaching the target or hitting obstacles. The plan at the box square is resolved at step 1 tick 2 using a Winner-Takes-All (WTA) mechanism. The average WTA kernel weights (bottom-left figure, averaging over $W_{\cdot h_1}$ and $W_{\cdot h_2}$ for $\cdot \in \{i, j, f, o\}$) subtract each channel from all the others, which through a sigmoid approximates an argmax. The magnitude of the diagonal entries (stronger for down than right) break ties.

Figure 20: Two equally good paths from box to target. Both are explored bidirectionally; Linear Plan Extension (LPE) kernels propagate activation along the path direction, Turn Plan Extension (TPE) kernels handle corners, and the Winner-Takes-All (WTA) mechanism resolves the tie. (From Taufeeque et al. [20].)

Stage 2 — Plan Extension Kernels. The RNN weight matrices contain two types of specialised kernel (Figure 20). **Linear Plan Extension (LPE)** kernels propagate plan activations one square per tick along the channel's direction; both forward LPE (extending from boxes toward targets) and backward LPE (extending from targets toward boxes) coexist in the weights, implementing **bidirectional search** at the weight level and mechanistically confirming the probe-level observation from Section 5.2.1. **Turn Plan Extension (TPE)** kernels transfer activation between direction channels at corners (e.g. box-right to box-down). TPE kernels have smaller magnitude than LPE, encoding a preference for continuing straight over turning.

Stage 3 — Winner-Takes-All. When multiple plans compete for the same box (e.g. down-then-right vs. right-then-down), the network resolves the ambiguity via cross-channel inhibition among short-term path channels. The average WTA kernel weights form a matrix with positive diagonal (self-reinforcement, $\sim$0.5) and negative off-diagonal (mutual inhibition, $\sim$−0.5). Combined with the sigmoid activation in the j-gate, this approximates an **argmax**: each direction channel subtracts all others from itself. The slightly stronger channel's activations suppress the weaker within 1–2 ticks.

The WTA mechanism undergoes a sharp **phase transition** during training transitions from (no inhibition) to (full WTA structure) at $\sim$19M training steps — less than 1% of total training (Figure 21). After this transition, the internal planning algorithm is qualitatively different: it has a selection mechanism that chooses between competing plans.

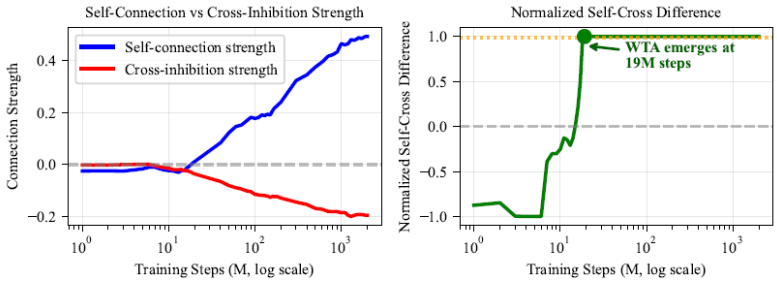


Figure 19: Emergence of the Winner-Takes-All mechanism during training. **Left:** Self-connection strength (blue) increases while cross-inhibition strength (red) becomes increasingly negative over training. **Right:** The normalized self-cross difference transitions sharply from -1 to $+1$ around 19M environment steps, indicating rapid emergence of WTA structure.

Figure 21: WTA phase transition during training. Left: self-connection strength increases while cross-inhibition becomes increasingly negative. Right: the normalised self-cross difference transitions sharply from -1 to $+1$ at $\sim$19M steps. (From Taufeeque et al. [20].)

STOPPING AND BACKTRACKING. Encoder kernels contribute *negative* activations at walls and obstacles, preventing plan extension through invalid squares — constituting a learned **transition model**. This stopping mechanism also enables **backtracking**: when a forward chain hits a wall, the *o*-gate generates a strong negative activation at the wall square. Because both forward and backward LPE kernels coexist, this negative activation propagates *backward* along the explored path via the backward LPE kernel, weakening the entire path segment. The weakened path then loses to any competing alternative via WTA. Causal verification confirms: forcing the negative wall activation to be positive (by taking its absolute value) blocks backtracking entirely — the network incorrectly attempts to chain through the wall.

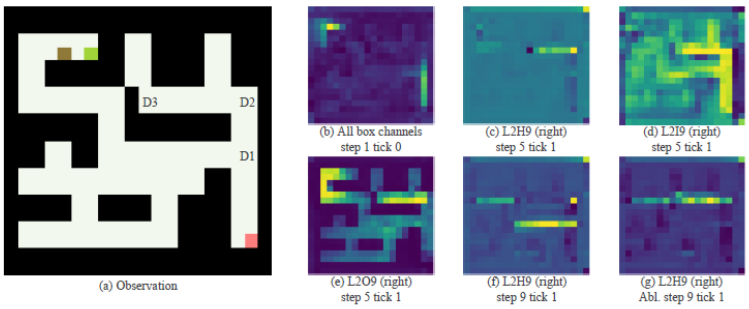


Figure 16: **(a)** 20×20 level we term as the "backtrack level" with key decision nodes D1-D3 for backward chaining. **(b)** The sum of box-movement channels at step 1 tick 0 indicates forward (from box) and backward (from target) chaining. **(c-e)** Activation of the box-right channel L2H9 involved in backward chaining at step 5 tick 1. Backward chaining moved up from D1 to D2 and then hitting a wall at D3, which initiates backtracking towards D2 through negative plan extension. The negative wall activation comes from the *o*-gate of L2H9. **(f)** Successful pathfinding at T28 after backtracking redirected the search. **(g)** Ablation: Forcing positive activation at D3 (by setting it to its absolute value) prevents backtracking, hindering correct solution finding (L2H9 Abl., T28).
In particular, the forced positive ablation at D3 results in an incorrect plan (g) which seemingly goes right all the way through the wall, as opposed to the correct plan (f) which goes right on a valid path.

Figure 22: Backtracking on a 20×20 level. A forward chain hits a wall, generating a negative activation that propagates backward via the backward LPE kernel, weakening the failed path. Forcing the wall activation positive (ablation) blocks backtracking entirely. (From Taufeeque et al. [20].)

SHORT-TERM AND LONG-TERM CHANNELS. Path channels decompose into **short-term** ($\sim$0–10 steps ahead) and **long-term** ($\sim$10–50 steps ahead) channels. Multiple channels per direction across layers enable faster spatial propagation (one square per layer per tick), while the short/long split handles temporally overlapping plans: when two boxes must pass through the same square at different times ($t_1 < t_2$), the long-term channel stores the later action while the short-term channel handles the immediate one.

INTERNAL VALUE FUNCTION DIVERGENCE. Path channel activations resemble a Q-function: targets anchor positive activation, obstacles generate negative activation, and the WTA uses activation strength to choose between plans. But the internal "value" dif-

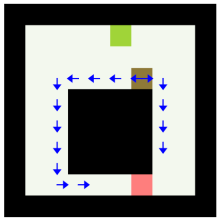


Figure 17: A level with two paths, one longer than the other. We initialize the starting hidden state with the two paths shown such that they both have two squares left to reach the target. We find that the expands both paths and picks the left (longer) path through the winner-takes-all mechanism since it reaches there with higher activation through linear-plan-extension.

fers from the training reward.The training reward includes a -0.1 per-step penalty, so longer plans should have lower cumulative value. But path channel activations do *not* capture plan length. Instead, shorter plans win because the LPE kernels extend plans one square per tick — so shorter paths from box to target have their forward and backward chains **connect sooner** (accumulating more ticks of activation) and therefore have stronger activations by the time the WTA resolves. The agent prefers shorter paths, but not for the reward-theoretic reason ($-0.1 \times$ length): it prefers them because of the dynamics of the LPE mechanism (shorter $\Rightarrow$ connects first $\Rightarrow$ wins WTA). This is verified by constructing levels where the shorter path has more remaining squares to connect than the longer path. In these cases, the agent takes the *longer* path — because it connects first despite being longer (Figure 23).

This is a **concrete, weight-level demonstration of mesa-optimisation**. The base optimiser (RL training with an explicit step penalty) produced an inner optimiser (bidirectional search with WTA) that has its own internally-defined value function (connection-time priority) that **differs from the training objective** the inner optimiser has a discoverable objective that was not specified by the outer optimiser.

5.3 Meta-RL and Mesa-Optimization

Is mesa-optimisation a general phenomenon? The meta-reinforcement learning (meta-RL) framework answers affirmatively: when a recurrent network is trained by RL across a *distribution* of tasks, its activation dynamics learn to implement a second RL algorithm specialised to that distribution.

THE META-RL SETUP. Wang et al. [23] and Duan et al. [24] train an RNN by standard RL across episodes, sampling a fresh MDP $m \sim \mathcal{D}$ each episode. The network receives the observation, previous action, and previous reward:

$$h_t = f_\theta(h_{t-1},\ [o_t,\ a_{t-1},\ r_{t-1}])$$

The weights θ are tuned slowly across many episodes by the outer RL algorithm. But because the network receives reward as input and retains state across timesteps within an episode, its *activation dynamics* can implement trial-by-trial learning: updating an internal policy based on observed rewards, without any weight change. After sufficient training, the weights can be **frozen** and the network still adapts to new MDPs from $\mathcal{D}$ — all learning occurs in the hidden state h_t. What emerges is a system with two nested RL algorithms:

- **Outer algorithm** (slow): the standard RL procedure (A2C/TRPO) that tunes θ across episodes. Model-free, general-purpose, data-hungry.
- **Inner algorithm** (fast): the learned RL procedure implemented in the activation dynamics of the trained RNN. Operates with frozen weights, adapts within episodes, specialised to $\mathcal{D}$.

The inner algorithm can differ from the outer in arbitrary ways: its effective learning rate, exploration strategy, and even its model-based versus model-free character can all diverge from the outer algorithm's.

EMERGENT MODEL-BASED BEHAVIOUR. On the Daw et al. [25] two-step task — a sequential decision task where a first-stage choice leads probabilistically (70/30) to one of two second-stage states — the meta-RL agent, trained entirely by model-free A2C, displays the canonical **model-based** choice pattern: after a rare-transition reward, it switches first-stage actions, taking the transition structure into account.

PLANNING AS INFERENCE-TIME COMPUTE. Jensen et al. [26] train a meta-RL agent (GRU) that can choose between physical actions and on-policy **rollouts** — imagined action sequences sampled from its own policy via a learned world model. The agent meta-learns *when* rollouts are beneficial: they are more frequent early in a trial and far from the goal. All networks transition from planning-heavy to reflexive behaviour as training progresses, and smaller networks plan more than larger ones — planning compensates for limited capacity. Mattar & Lengyel [27] find convergent evidence that biological planning is focused, depth-limited, and serial. This connects to mesa-optimisation: the inner algorithm learns not only *what* to compute but *when* to engage planning, and inference-time compute is allocated where the current objective benefits, not where the true objective would.

5.4 Planning as Probabilistic Inference

Solway & Botvinick [28] propose that goal-directed decision making is not reward maximisation but **Bayesian inversion of a generative model of reward**. The agent constructs a probabilistic graphical model over states S, actions A, policies Π, successor states S', and a reward node $\hat{R}$, linked by the transition function $p(s' \mid s, a)$.

BINARY REWARD ENCODING. Reward is encoded as a binary variable $\hat{r} \in \{0, 1\}$, with magnitude represented as the probability $p(\hat{r} = 1 \mid s')$ (e.g. via a linear rescaling to $[0, 1]$). This is essential: one cannot condition on a scalar reward to perform abductive inference, but one *can* condition on the binary event $\hat{r} = 1$. The decision problem becomes likelihood maximisation: $\arg\max_\pi p(\hat{r} = 1 \mid s, \pi)$.

POLICY ABDUCTION AND ITERATIVE INFERENCE. The likelihood $p(\hat{r} \mid s, \pi) = \sum_a p(a \mid s, \pi) \sum_{s'} p(s' \mid s, a)\, p(\hat{r} \mid s')$ equals the expected reward under π. *Inverting* the model via Bayes' rule gives the posterior over policies:

$$p(\pi \mid s, \hat{r} = 1) \propto p(\hat{r} = 1 \mid s, \pi)\, p(\pi)$$

Single-step abduction can fail with non-uniform priors or sequential plans, so the inference is *iterated*: on each iteration n, the observation $\hat{r} = 1$ is reinstated and the posterior updated:

$$p_{n+1}(\pi \mid s, \hat{r}) \propto p(\hat{r} \mid s, \pi)\, p_n(\pi \mid s, \hat{r})$$

With a uniform initial prior, this yields $p_n(\pi \mid s, \hat{r}) \propto p(\hat{r} \mid s, \pi)^n$ — a Boltzmann policy with temperature $1/n$, initially diffuse and gradually concentrating on the optimum:

$$p_n(\pi \mid s, \hat{r}) \propto p(\hat{r} \mid s, \pi)^n \tag{13}$$

Solway & Botvinick prove **monotonic convergence**: $p_n(\pi^* \mid s, \hat{r}) \to 1$ as $n \to \infty$, with the log posterior ratio growing linearly at the log-likelihood ratio (equivalent to the sequential probability ratio test). In multi-step settings, convergence propagates backward from terminal states.

STRUCTURAL PROPERTIES. Four properties distinguish this from standard reward maximisation:

1. **Bounded reward**: the binary encoding caps reward influence at $[0, 1]$, preventing unbounded density ratios that enable heavy-tailed exploitation (cf. Kwa et al., Section 3.2.2).
2. **Distributional output**: the posterior is a distribution over policies, not a single argmax — the agent does not collapse to a single exploitative policy.
3. **Prior regularisation**: the prior $p(\pi)$ enters multiplicatively at every iteration, functioning as an implicit regulariser without an explicit KL penalty.
4. **Gradual convergence**: the Boltzmann temperature $1/n$ decreases slowly — the agent approaches the optimum along a smooth interior path rather than jumping to an extreme policy.

6 RESEARCH DIRECTIONS

We outline research directions on goal-directed learning organised into three groups.

6.1 Formal Reward Learning and Identifiability

The projected angle $\angle(\vec{R}_0, \vec{R}_1)$ (Section 3.1.1) and the overoptimisation coefficient β_{RL} (Section 3.1) both measure how much room the proxy has to diverge from truth. The partial identifiability results (Section 2.2) show that training data constrains the proxy only up to an equivalence class $[R^*]_f$. This direction makes the connection between these quantities explicit.

6.1.1 Partial Identifiability and Goal Misgeneralisation

Does the size of the equivalence class quantitatively reflect how severe is GM, and how fast overoptimisation will occur?

PROJECTED ANGLE DIAMETER Define the **projected angle diameter** Θ_f which is the bound θ in Theorem 1 (optimal stopping, Eq. 6).

$$\Theta_f(\mathcal{D}) = \max_{R' \in [R^*]_f} \angle(\vec{R}^*, \vec{R}') = \max_{R' \in [R^*]_f} \measuredangle(M_\mathcal{T} \vec{R}^*, M_\mathcal{T} \vec{R}') \tag{14}$$

where $[R^*]_f$ is the set of rewards consistent with training data $\mathcal{D}$ under data source f. [15] The data source f can be any of those characterised in Section 2.2: demonstrations, trajectory comparisons, or direct observation of rollouts under a known training policy.

15 The projection $M_\mathcal{T}$ quotients out potential shaping (which lives in $\ker(M_\mathcal{T})$), and the angle is scale-invariant, so positive linear scaling is irrelevant.

COMPUTING Θ_f. The computation is most tractable when the transition dynamics $\mathcal{T}$ are known, allowing exact computation of $M_\mathcal{T} = I - \mathbf{A}^T(\mathbf{A}\mathbf{A}^T)^{-1}\mathbf{A}$. For a data source that observes noiseless trajectory comparisons, $[R^*]_f$ projected onto span(Ω) forms a convex cone — the intersection of half-spaces $M_\mathcal{T}\vec{R}' \cdot (\vec{\eta}^{\tau_w} - \vec{\eta}^{\tau_l}) \geq 0$. The diameter Θ_f is the maximum angle between any two rays in this cone, which is a second-order cone program (SOCP):

1. Project onto span(Ω) via $M_\mathcal{T}$
2. Data-source constraints become linear: e.g. $M_\mathcal{T}\vec{R}' \cdot (\vec{\eta}^{\tau_w} - \vec{\eta}^{\tau_l}) \geq 0$ for preference data, or $\arg\max_a Q_{R'}(s,a) = a^*$ for demonstration data
3. Normalise to $\|M_\mathcal{T}\vec{R}'\| = 1$ and maximise $\measuredangle(M_\mathcal{T}\vec{R}^*, M_\mathcal{T}\vec{R}')$

HYPOTHESIS. Θ_f affects both GM and overoptimisation: (i) OOD test reward $J_{R^*}(\pi_{\text{OOD}})$ is a monotonically decreasing function of Θ_f, and (ii) the Gao et al. overoptimisation rate β_{RL} scales monotonically with Θ_f. The experiment determines the functional form empirically.

EXPERIMENT 1: Θ_f PREDICTS GM SEVERITY. In a tabular gridworld with training diversity α (fraction of levels with randomised goal placement, mirroring the Langosco et al. sweep), compute $\Theta_f(\alpha)$ exactly via the SOCP and measure OOD test reward. Fit the relationship between Θ_f and OOD reward degradation — determining whether it is linear, threshold-like, or sublinear.

EXPERIMENT 2: Θ_f PREDICTS OVEROPTIMISATION RATE. Using Karwowski et al.'s tabular MDP suite, vary the projected angle $\measuredangle(\vec{R}_0, \vec{R}_1)$ and measure the Goodhart curve for each. Compute Θ_f from simulated training data and test whether Θ_f predicts β (the rate of true-reward decline past the Goodhart peak) more tightly than the raw projected angle alone.

RESULTS. If confirmed, partial identifiability (Skalse), Goodhart geometry (Karwowski), and empirical overoptimisation (Gao) are all measuring the same underlying quantity — the degree to which training data constrains the proxy within span(Ω).

6.1.2 Empirical Demonstration of Transfer Reward Hacking

Can we demonstrate reward hacking due to transfer impossibility in practice?

BACKGROUND. The transfer impossibility theorem (Theorem 1 in Section 2.2) proves that wherever training dynamics $\mathcal{T}_1$ differ from deployment dynamics $\mathcal{T}_2$, the learned reward is *completely unconstrained*: for any arbitrary target function $\mathcal{L}$, there exists a reward $R' \in [R^*]_f$ whose expected value under $\mathcal{T}_2$ equals $\mathcal{L}$.

EXPERIMENTAL SETUP.

1. Train a reward model R_ϕ on trajectory comparisons generated in environment $\mathcal{E}_1$ (e.g. a dialogue task with a specific user simulator that determines the transition dynamics $\mathcal{T}_1$).
2. Deploy R_ϕ in environment $\mathcal{E}_2$ — the same task structure but with a different user simulator producing different state transitions $\mathcal{T}_2$.
3. Optimise a policy π against R_ϕ in $\mathcal{E}_2$ via PPO. Measure three quantities:
   - **Proxy reward** $J_{R_\phi}(\pi)$ (should rise monotonically — the policy is optimising the proxy)
   - **True quality** $J_{R^*}(\pi)$ (should degrade — the proxy is unconstrained on OOD transitions)
   - **Location of degradation**: for each state-action pair (s,a), compute $|\mathcal{T}_1(s,a) - \mathcal{T}_2(s,a)|$ and the local proxy–true divergence $|R_\phi(s,a) - R^*(s,a)|$. The theorem predicts these should be correlated: degradation concentrates where dynamics differ.
4. Vary the magnitude of the dynamics shift $\|\mathcal{T}_1 - \mathcal{T}_2\|_{\text{TV}}$ (total variation distance between transition kernels) by interpolating between user simulators. For each shift magnitude, measure the peak and asymptotic gold reward under PPO optimisation.

RESULTS. The plot of gold reward peak versus dynamics shift $\|\mathcal{T}_1 - \mathcal{T}_2\|_{\text{TV}}$ should yield a smooth degradation curve — an empirical scaling law for transfer-induced reward hacking. The theorem predicts complete unconstrained-ness in the worst case, so the empirical question is how much of the theoretical worst case is realised by practical neural RMs. If the degradation curve is approximately linear, transfer hacking is a quantitative (manageable) risk; if it is catastrophic beyond a threshold, it identifies a hard boundary on how much dynamics shift is tolerable for a given RM.

6.1.3 Planning and the Occupancy Polytope

Does a neural network's plans (extracted from weights) reflect the trajectory through the occupancy polytope? The DRC's converged plan (readable from weights) determines a planned trajectory, which in turn determines an occupancy measure $\vec{\eta}^{\pi} \in \Omega$

- **Plan-formation trajectories through Ω.** Read $\vec{\eta}^{\pi}$ from path channels at each internal tick as the plan forms. Track the resulting trajectory through Ω and compare to Karwowski et al.'s steepest ascent prediction. This tests whether neural plan formation follows the polytope geometry derived for idealised policy optimisation.
- **Implicit reward extraction.** Given the agent's converged policy (a vertex of Ω), the set of rewards for which this policy is optimal forms a cone in $\mathrm{span}(\Omega)$. The centre of this cone is the agent's "implicit reward." Computing the projected angle between this implicit reward and the training reward would provide a geometric quantification of mesa-optimisation

6.2 Planning Representations and Interpretability

The DRC interpretability programme (Sections 5.1–5.2) revealed three separable components of the agent's planning algorithm:

1. **Goal representation**: the squares from which backward LPE chains originate. In standard Sokoban, encoder kernels detect the red-pixel target feature and seed backward chains from it. The goal is operationally whatever the backward chains treat as their origin.
2. **Learned validity constraints**: the stopping mechanism (negative activations at walls from encoder and entity channels) that prevents plan extension through obstacles. The LPE kernels themselves trivially generalise — they just shift activation one square — but the stopping conditions encode which transitions are valid and may not.
3. **Selection criterion**: connection-time priority via WTA. Already characterised in Section 5.2.2

6.2.1 Extracting and Testing the Goal Representation

1. *Can we read off the proxy goal from the path channels, and does this predict GM?*
2. *Instead of using an RNN of fixed N ticks, let the agent decide how many ticks to perform by making no-op an explicit action during training. How does the agent choose how long to think for? It must know when the goal is reached internally — before the actual goal is reached — in order to stop taking no-ops.*
3. *Humans have generic planning algorithms that make us sample-efficient at solving novel problems, while RNNs need more samples to form better plans. Can the planning algorithms in RNNs generalise to other games? The spatial structure of the convolutions is task-specific, but can we adapt a spatial head to the planning head?*

BACKGROUND. The encoder kernels are convolutional and spatially invariant — they cannot encode "upper-right" in their weights. But the pool-and-inject mechanism (a weighted combination of mean- and max-pooled activations injected at each step) provides a global spatial signal, and the recurrent dynamics can learn position-dependent biases through the interaction of pooled signals with local activations. If targets always appear in a fixed region during training, the backward-chain initialisation could become contingent on this global spatial signal rather than on the local red-pixel feature. The backward-chain origin map — where in the grid backward LPE chains start — directly reveals which cue the agent uses.

EXPERIMENTAL SETUP.

1. **Training.** Train DRC(3,3) on modified Sokoban where targets are restricted to a fixed spatial region (e.g. upper-right quadrant) during training, mirroring the Langosco et al. CoinRun design. Vary the fraction α of levels with randomised target placement.
2. **Backward-chain origin map.** At each training checkpoint and each α, run the agent on held-out levels with targets at novel locations. Decode path channels and record where backward chains originate. If they track the target's actual position, the agent has learned the entity feature; if they cluster in the training region regardless, it has learned the spatial proxy.
3. **WTA phase transition timing.** Taufeeque et al. [20] showed that the WTA mechanism undergoes a sharp phase transition. At each training checkpoint, record both $\mathrm{WTA}_{\mathrm{norm}}$ and the backward-chain origin map. This reveals whether goal representation forms *before*, *during*, or *after* the WTA transition — distinguishing whether the planning algorithm inherits a pre-existing goal (goal forms first), or the planning selection dynamics shape which goal survives (WTA forms first).

RESULTS.

- At $\alpha = 0$ (pure spurious correlation), backward chains should anchor to the spatial region. At $\alpha = 1$ (no correlation), they should track the red-pixel feature. Because the encoder kernels are convolutional (spatially invariant), the red-pixel detector is always available in the weights — the spatial proxy can only dominate via the pool-and-inject pathway, which requires consistent spatial statistics. Even a small fraction of randomised levels disrupts the pooled statistics, so the transition from spatial proxy to entity feature should occur at small α. How sharp this transition is — whether it resembles a phase transition or a gradual crossover — is an empirical question that the backward-chain origin map can answer directly.
- If the goal representation forms *after* the WTA phase transition, the planning mechanism's selection dynamics (connection-time priority via WTA) play an active role in determining which goal representation stabilises — implicating the architecture's planning inductive bias in goal formation, not just the training data.

6.2.2 Planning, Mesa-Optimisation, and Reward Hacking

Does the inner optimiser's objective diverge from the training reward in a quantifiable way? Does planning enable proactive detection of reward hacking?

STEP PENALTY INVARIANCE AS A MESA-OPTIMISATION TEST. Taufeeque et al. [20] showed that the DRC's internal value function does not encode the -0.1 per-step penalty: shorter paths win via connection-time priority (whichever plan's forward and backward LPE chains connect first accumulates more ticks of activation and wins WTA), not via discounted reward. This is a qualitative observation of mesa-optimisation — the inner optimiser has a discoverable objective that differs from the training signal. To quantify this behaviour:

1. **Training.** Train DRC(3,3) with step penalties $c \in \{0, -0.01, -0.1, -0.5, -1.0\}$, keeping all other reward components fixed ($+10$ for solving, $+1$ for box-on-target).
2. **WTA selection criterion.** For each c, test on adversarial levels where the shortest path $\neq$ the fastest-connecting path (following the design of Taufeeque et al.'s Figure 17, where two paths have equal remaining squares to connect but different total lengths). Measure whether WTA selects the mesa-optimal path (fastest-connecting) or the training-reward-optimal path (shortest under step penalty c).
3. **Mesa-optimisation gap.** Define the gap as the fraction of adversarial levels where the agent's choice differs from the training-reward-optimal choice. If connection-time priority is invariant to c, this gap grows with $|c|$ — providing a quantitative measure of mesa-optimisation severity.
4. **Weight-level verification.** Check whether LPE kernel magnitudes or WTA connectivity patterns change across step penalties. Invariance would confirm that the inner optimiser's structure is architecturally determined, not shaped by the reward signal.

DOES REWARD HACKING APPEAR IN PLANS? MacDiarmid et al. showed reward hacking during RL training causes broad emergent misalignment. The DRC's readable plans allow a direct test: *does the exploit appear in the plan before it appears in behaviour?* If planning precedes hacking, plan auditing (reading path channels) can detect exploits proactively. If hacking precedes planning, the exploit is discovered by policy gradient and cached into the plan post-hoc.

GENERALITY ACROSS ARCHITECTURES. All results above depend on the ConvLSTM's spatial structure making plans directly readable. A key open question is whether similar representations emerge in other architectures. Training a **Transformer-based** agent (causal transformer on tokenised grid states) on Sokoban and applying the Bush/Taufeeque diagnostic suite (linear probes for C_A/C_B, causal interventions) would test whether the interpretability toolkit transfers or whether it is specific to the ConvLSTM's spatial inductive bias.

6.3 Neuroscientific Frameworks for RL Agents

6.3.1 EVB Predicts Which Proxy Goal an Agent Will Learn

HYPOTHESIS. Section 4.2 argued that distinguishing states have low EVB under narrow training. Can EVB quantitatively predict which specific proxy an agent will adopt? In an environment with multiple candidate proxy goals (e.g. spatial position, colour, and object type all correlated with the true goal during training), the proxy whose distinguishing states have the **lowest total EVB** should be the one that survives training, as those are the states the agent is least likely to learn about.

FORMAL PREDICTION. For each candidate proxy $\tilde{R}_j$, define the set of **distinguishing states** $\mathcal{D}_j = \{s : \tilde{R}_j(s) \neq R^*(s)\}$ — states where proxy j disagrees with truth. The **Need-only** predictor uses only the state-visitation component of EVB:

$$\text{Need}_j(s) = \sum_{s_k \in \mathcal{D}_j} M^{\pi_{\text{train}}}(s, s_k)$$

This measures the total expected visitation of proxy j's distinguishing states. Its advantage is that it depends only on the policy's state-visitation structure (the SR), which is observable without knowing the reward, and does not assume experience replay. The **full EVB** predictor additionally weights each state by the Gain term:

$$\mathrm{EVB}_j(s) = \sum_{s_k \in \mathcal{D}_j} \sum_{a_k} \mathrm{Gain}(s_k, a_k) \times M^{\pi_{\text{train}}}(s, s_k)$$

where Gain is computed using the true reward R^*. Gain captures whether a backup at s_k would actually change the policy there: a state where Q-values under R^* are nearly tied has high Gain (a backup could flip the policy), while one where a single action dominates has low Gain (the backup changes nothing). Two distinguishing states with equal Need but different Gain are distinguished by EVB but not by Need alone. In both cases, the prediction is: the agent adopts proxy $j^* = \arg\min_j \mathrm{EVB}_j$ (or $\arg\min_j \mathrm{Need}_j$) — the proxy whose distinguishing evidence receives the least learning signal.

EXPERIMENTAL SETUP.

1. Design a family of MiniGrid environments with three candidate proxies: (a) spatial position (goal always in the upper-right quadrant), (b) colour (goal always on a blue tile), (c) object type (goal is always a diamond). During training, all three correlate perfectly with the true goal.
2. Train agents with both on-policy (PPO) and off-policy (DQN) algorithms. Compute the SR $M^{\pi_{\text{train}}}$ from the trained policy's state occupancies, and compute both Need_j and EVB_j for each proxy.
3. At test time, decorrelate each proxy separately (e.g. move the goal to the lower-left on a blue tile — this breaks the spatial proxy while preserving colour and object). Measure which proxy the agent continues to follow.
4. Compare the observed proxy selection against the Need prediction and the full EVB prediction.

WHY THIS MATTERS. If EVB (or Need) correctly predicts proxy selection, it provides a predictive theory of which proxy goal an agent will learn — not just that GM will occur, but what form it will take.

6.3.2 Planning-as-Inference vs Reward Maximisation

SETUP. The planning-as-inference framework (Section 5.4) produces a family of policies parametrised by iteration count n: $\pi_n(a \mid s) \propto p(\hat{r} \mid s, \pi)^n$ (Eq. 13). This is directly comparable to the family $\{\pi_\lambda\}_{\lambda \in [0,1]}$ used by Karwowski et al. (Section 3.1.2) to parametrise optimisation pressure via MCE or Boltzmann rationality. In both cases, the agent interpolates from uniform ($n = 1$ or $\lambda = 0$) to deterministic ($n \to \infty$ or $\lambda = 1$). The key difference is the *trajectory through the occupancy polytope* Ω. Define the inference agent's occupancy trajectory as $\vec{\eta}_n = \vec{\eta}^{\pi_n}$ and the optimisation agent's trajectory as $\vec{\eta}_\lambda = \vec{\eta}^{\pi_\lambda}$, both starting from the same interior point $\vec{\eta}_{\text{unif}}$. At each point along the trajectory, the true reward changes at rate $\frac{d}{dn} J_{R_0}(\pi_n) = \vec{t}_n \cdot M_{\mathcal{T}} \vec{R}_0$, where $\vec{t}_n$ is the unit tangent. Goodharting occurs when $\vec{t}_n \cdot M_{\mathcal{T}} \vec{R}_0 < 0$ while $\vec{t}_n \cdot M_{\mathcal{T}} \vec{R}_1 > 0$ — the proxy still improves but the true reward declines.

HYPOTHESIS. The inference agent's trajectory $\{\vec{\eta}_n\}$ should exhibit **later onset and lower severity** of Goodharting than the optimisation agent's trajectory $\{\vec{\eta}_\lambda\}$, due to three structural properties of the inference framework that each address a specific failure mode identified earlier in this article.

Mechanism 1: Bounded per-step movement (addresses heavy-tailed exploitation, Section 3.2.2). The Boltzmann posterior $\pi_n \propto p(\hat{r} \mid s, \pi)^n$ changes by a multiplicative factor of $p(\hat{r} \mid s, \pi)$ per iteration. Since $p(\hat{r} \mid s, \pi) \in [0, 1]$, the per-step density ratio is bounded:

$$\frac{\pi_{n+1}(a \mid s)}{\pi_n(a \mid s)} \leq \frac{1}{\min_{a'} p(\hat{r} \mid s, a')}$$

This directly bounds the per-step χ^2 divergence $D_{\chi^2}(\mu_{\pi_{n+1}} \| \mu_{\pi_n})$. By the Laidlaw et al. bound (Theorem 5), the true reward degradation per step is controlled by $\sqrt{(1 - r^2) \cdot D_{\chi^2}(\mu_{\pi_{n+1}} \| \mu_{\pi_n})}$. The binary reward encoding caps the likelihood ratio, preventing the unbounded density ratios that Kwa et al. showed KL cannot control.

Mechanism 2: Implicit occupancy-measure regularisation (addresses AD insufficiency, Theorem 5). After n iterations from uniform prior, the cumulative occupancy shift satisfies:

$$D_{\chi^2}(\mu_{\pi_n} \| \mu_{\pi_{\text{unif}}}) = \sum_{s,a} \frac{(\mu_{\pi_n}(s,a) - \mu_{\pi_{\text{unif}}}(s,a))^2}{\mu_{\pi_{\text{unif}}}(s,a)}$$

Because π_{unif} has full support, $\mu_{\pi_{\text{unif}}}(s,a) > 0$ everywhere, so D_{χ^2} is always finite. The prior $p(\pi) = \pi_{\text{unif}}$ acts as an implicit OM regulariser, penalising exactly the occupancy-measure shift that Laidlaw et al. proved AD regularisation cannot control.

Mechanism 3: Extended interior-phase duration (addresses boundary-deflection Goodharting, Section 3.1.3). In the Karwowski geometry, the interior phase (where $\vec{t}_i \cdot M_T \vec{R}_0 > 0$) is safe; Goodharting begins only at boundary faces. The inference trajectory stays interior longer because Boltzmann annealing ($1/n$) approaches the boundary *asymptotically*, whereas unconstrained gradient ascent reaches boundary faces in finite steps. The inference trajectory satisfies concavity (5) with *equality* (each step rescales the same likelihood vector), while steepest ascent satisfies it with strict inequality after each boundary hit — delaying the onset of the boundary-deflection mechanism that causes Goodharting.

EXPERIMENTAL SETUP. Test on the Karwowski et al. MDP suite (30,400 configurations). Compare the MCE optimisation agent (π_α swept over α) against the Boltzmann inference agent ($\pi_n \propto p(\hat{r} \mid s, \pi)^n$ swept over n). For each, compute occupancy measures and evaluate both true and proxy reward. Measure Goodhart onset (normalised to common KL scale), NDH at full optimisation, and the occupancy trajectory through Ω.

PREDICTIONS.

- $n^* > \alpha^*$ (later onset) at matched proxy KL, because bounded per-step density ratios prevent early boundary hits.
- $|\mathrm{NDH}_{\mathrm{inf}}| < |\mathrm{NDH}_{\mathrm{opt}}|$ (lower severity), because implicit OM regularisation controls the occupancy shift driving proxy–true decorrelation.
- At $\gamma = 0$, $\mathrm{NDH}_{\mathrm{inf}} = \mathrm{NDH}_{\mathrm{opt}}$ (OM = AD in the bandit limit). The gap should *widen* monotonically as $\gamma \to 1$, where cascading state-visitation drift dominates.

RESULTS A positive result would establish that optimisation framing itself — not just the specific algorithm — is a root cause of reward hacking, and that the structural properties of inference (bounded reward, distributional output, prior regularisation) provide a principled alternative to post-hoc KL penalties.

6.3.3 *Characterizing Goal Misgeneralisation with the MB/MF/SR Failure*

BACKGROUND. Trinh et al. showed that misgeneralising CoinRun agents do not represent the coin at all — but "no coin representation" conflates structurally distinct failure modes. The revaluation typology (Section 4.1) distinguishes pure MF, SR, and MB failures by their response to reward devaluation, transition changes, and policy revaluation — providing a clean diagnostic when transplanted from human participants to RL agents.

EXPERIMENTAL SETUP. Train three agent types on a MiniGrid (12×12) environment where the goal object is always in a fixed region during training:

- **SR-TD agent**: PPO with the SR as its state representation, storing M^π via the vector TD rule (Eq. 11) and learning reward weights w separately.
- **Pure MF agent**: standard PPO with one-hot state features — equivalent to the Langosco et al. CoinRun architecture.
- **SR-MB agent**: learns a one-step transition model T^π and recomputes $M^\pi = (I - \gamma T^\pi)^{-1}$ at decision time.

At test time, apply three manipulations:

1. **Reward devaluation**: keep the goal at the training location but set its reward to zero. The SR-TD agent should immediately reduce its value estimates ($V = M^\pi \cdot w_{\mathrm{new}}$ with $w_{\mathrm{new}} = 0$ at the goal); the pure MF agent should continue pursuing the cached policy.
2. **Transition change**: move the goal to a new location. Both MF and SR-TD agents should fail; the SR-MB agent should adapt by recomputing M^π from the updated T^π.
3. **Policy revaluation**: introduce a large new reward at a location the trained policy never visits. SR-MB should fail (its T^π is built from the old policy that avoids the new reward); only an agent with off-policy replay (SR-Dyna) should succeed.

PROBE-BASED VERIFICATION. For each agent, train linear probes on hidden states predicting (i) future state occupancy $\hat{M}(s, s')$ and (ii) goal identity. MF agents should show near-zero probe accuracy for future occupancy; SR agents should show high accuracy. The goal-identity probe distinguishes SR failure from MB success.

REFERENCES

[1] Rohin Shah, Vikrant Varma, Ramana Kumar, Mary Phuong, Victoria Krakovna, Jonathan Uesato, and Zac Kenton. Goal misgeneralization: Why correct specifications aren't enough for correct goals, 2022.

[2] Lauro Langosco, Jack Koch, Lee Sharkey, Jacob Pfau, Laurent Orseau, and David Krueger. Goal misgeneralization in deep reinforcement learning, 2023.

[3] Joar Skalse, Matthew Farrugia-Roberts, Stuart Russell, Alessandro Abate, and Adam Gleave. Invariance in policy optimisation and partial identifiability in reward learning, 2023.

[4] Joar Skalse and Alessandro Abate. Misspecification in inverse reinforcement learning, 2023.

[5] Monte MacDiarmid, Benjamin Wright, Jonathan Uesato, Joe Benton, Jon Kutasov, Sara Price, Naia Bouscal, Sam Bowman, Trenton Bricken, Alex Cloud, Carson Denison, Johannes Gasteiger, Ryan Greenblatt, Jan Leike, Jack Lindsey, Vlad Mikulik, Ethan Perez, Alex Rodrigues, Drake Thomas, Albert Webson, Daniel Ziegler, and Evan Hubinger. Natural emergent misalignment from reward hacking in production rl, 2025.

[6] Jan Betley, Daniel Tan, Niels Warncke, Anna Sztyber-Betley, Xuchan Bao, Martín Soto, Nathan Labenz, and Owain Evans. Emergent misalignment: Narrow finetuning can produce broadly misaligned llms, 2026.

[7] Anna Soligo, Edward Turner, Senthooran Rajamanoharan, and Neel Nanda. Emergent misalignment is easy, narrow misalignment is hard, 2026.

[8] Jacek Karwowski, Oliver Hayman, Xingjian Bai, Klaus Kiendlhofer, Charlie Griffin, and Joar Skalse. Goodhart's law in reinforcement learning, 2023.

[9] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization, 2022.

[10] Thomas Kwa, Rachel Thomas, and Adrià Garriga-Alonso. Catastrophic goodhart: Regularizing RLHF with KL divergence does not mitigate heavy-tailed reward misspecification, 2024.

[11] Cassidy Laidlaw, Shivam Singhal, and Anca Dragan. Correlated proxies: A new definition and improved mitigation for reward hacking, 2025.

[12] Audrey Huang, Wenhao Zhan, Tengyang Xie, Jason D. Lee, Wen Sun, Akshay Krishnamurthy, and Dylan J. Foster. Correcting the mythos of KL-regularization: Direct alignment without overoptimization via chi-squared preference optimization, 2024.

[13] Jian Shi, Ziheng Song, Dongruo Zhou, Quanquan Zhang, Maryam Fazel, and Simon S. Du. Understanding the performance gap in preference learning: A dichotomy of RLHF and DPO, 2025.

[14] Nathaniel D Daw, Yael Niv, and Peter Dayan. Uncertainty-based competition between prefrontal and dorsolateral striatal systems for behavioral control. *Nature Neuroscience*, 8(12):1704–1711, November 2005.

[15] Ray J. Dolan and Peter Dayan. Goals and habits in the brain. *Neuron*, 80(2):312–325, October 2013.

[16] Evan M. Russek, Ida Momennejad, Matthew M. Botvinick, Samuel J. Gershman, and Nathaniel D. Daw. Predictive representations can link model-based reinforcement learning to model-free mechanisms. *PLOS Computational Biology*, 13(9):e1005803, September 2017.

[17] Peter Dayan. Improving generalization for temporal difference learning: The successor representation. *Neural Computation*, 5(4):613–624, July 1993.

[18] Marcelo G. Mattar and Nathaniel D. Daw. Prioritized memory access explains planning and hippocampal replay. *Nature Human Behaviour*, 2:1273–1283, November 2018.

[19] Arthur Guez, Mehdi Mirza, Karol Gregor, Rishabh Kabra, Sébastien Racanière, Théophane Weber, David Raposo, Adam Santoro, Laurent Orseau, Tom Eccles, Greg Wayne, David Silver, and Timothy Lillicrap. An investigation of model-free planning, 2019.

[20] Mohammad Taufeeque, Aaron David Tucker, Adam Gleave, and Adrià Garriga-Alonso. Path channels and plan extension kernels: a mechanistic description of planning in a sokoban rnn, 2025.

[21] Thomas Bush, Stephen Chung, Usman Anwar, Adrià Garriga-Alonso, and David Krueger. Interpreting emergent planning in model-free reinforcement learning, 2025.

[22] Mohammad Taufeeque, Philip Quirke, Maximilian Li, Chris Cundy, Aaron David Tucker, Adam Gleave, and Adrià Garriga-Alonso. Planning in a recurrent neural network that plays sokoban, 2025.

[23] Jane X Wang, Zeb Kurth-Nelson, Dhruva Tirumala, Hubert Soyer, Joel Z Leibo, Remi Munos, Charles Blundell, Dharshan Kumaran, and Matt Botvinick. Learning to reinforcement learn, 2017.

[24] Yan Duan, John Schulman, Xi Chen, Peter L. Bartlett, Ilya Sutskever, and Pieter Abbeel. Rl^2: Fast reinforcement learning via slow reinforcement learning, 2016.

[25] Nathaniel D. Daw, Samuel J. Gershman, Ben Seymour, Peter Dayan, and Raymond J. Dolan. Model-based influences on humans' choices and striatal prediction errors. *Neuron*, 69(6):1204–1215, March 2011.

[26] Kristopher T. Jensen, Guillaume Hennequin, and Marcelo G. Mattar. A recurrent network model of planning explains hippocampal replay and human behavior. *Nature Neuroscience*, 27(7):1340–1348, June 2024.

[27] Marcelo G. Mattar and Máté Lengyel. Planning in the brain. *Neuron*, 110(6):914–934, March 2022.

[28] Alec Solway and Matthew M. Botvinick. Goal-directed decision making as probabilistic inference: A computational framework and potential neural correlates. *Psychological Review*, 119(1):120–154, 2012.